

Analiza MVCC mehanizma u PostgreSQL bazi podataka

Autor: Julija Ristic 1923

Predmetni profesor: prof. Aleksandar Stanimirovic

Predmet: Sistemi za upravljanje bazama podataka

1. Uvod
2. MVCC koncept
 - 2.1 Definicija i osnovni koncept
 - 2.2 Zašto se MVCC koristi
 - 2.3 Tipovi MVCC pristupa
 - 2.4 Osnovni principi rada
 - 2.5 Timestamp ordering protokol (formalni model)
3. PostgreSQL kao sistem za upravljanje bazama podataka
 - 3.1 Arhitektura PostgreSQL
 - 3.2 Upravljanje transakcijama u PostgreSQL-u
4. Implementacija MVCC mehanizma u PostgreSQL
 - 4.1 Verzionisanje slogova (Tuple Versioning)
 - 4.2 Interna struktura tuple zapisa
 - 4.3 Uloga xmin i xmax atributa
 - 4.4 Kreiranje novih verzija podataka
 - 4.5 Snapshot mehanizam
 - 4.6 Obrada konkurentnih transakcija
 - 4.7 Upravljanje konfliktima između transakcija
5. Izolacioni nivoi i MVCC
 - 5.1 Read Committed
 - 5.2 Repeatable Read
 - 5.3 Serializable Snapshot Isolation
 - 5.4 Poređenje izolacionih nivoa
6. Upravljanje prostorom i održavanje baze
 - 6.1 Data Bloat fenomen
 - 6.2 VACUUM mehanizam
 - 6.3 Autovacuum
 - 6.4 VACUUM FULL
7. Demonstracija MVCC mehanizma u PostgreSQL
 - 7.1 Inspekcija tuple zaglavlja i xmin/xmax atributa
 - 7.2 Demonstracija izolacionih nivoa u paralelnim sesijama
 - 7.3 Anomalije konkurentnog izvršavanja i MVCC zaštita
 - 7.4 Upravljanje mrtvim tuple-ovima i VACUUM analiza
 - 7.5 Merenje performansi i analiza planova izvršavanja
8. Prednosti i ograničenja MVCC pristupa
 - 8.1 Smanjenje blokiranja i poboljšanje konkurentnosti
 - 8.2 Konzistentni snimci i skalabilnost
 - 8.3 Optimizacija performansi kroz Visibility Map
 - 8.4 Potrošnja prostora i troškovi održavanja
 - 8.5 Dugotrajne transakcije i kompleksnost implementacije
9. Zaključak
10. Literatura

1. Uvod

Sistemi za upravljanje bazama podataka (DBMS) predstavljaju osnovu savremenih informacionih sistema i imaju ključnu ulogu u skladištenju, organizaciji i obradi podataka. Razvojem interneta, mobilnih aplikacija i sistema koji generišu velike količine podataka, zahtevi za pouzdanim i efikasnim upravljanjem podacima postali su značajno veći. Relacione baze podataka, zasnovane na relacionom modelu i dalje predstavljaju dominantno rešenje u poslovnim sistemima zahvaljujući svojoj pouzdanosti, zrelosti i mogućnosti očuvanja integriteta podataka.

Jedan od najvažnijih izazova sa kojima se sistemi za upravljanje bazama podataka suočavaju jeste istovremeni pristup podacima od strane velikog broja korisnika. U savremenim aplikacijama više transakcija se izvršava paralelno, pri čemu pojedini korisnici čitaju podatke dok ih drugi istovremeno menjaju. Bez odgovarajućih mehanizama kontrole mogu nastati različite anomalije, kao što su čitanje nepotvrđenih podataka, gubitak izmena ili nekonzistentni rezultati upita, što može ugroziti ispravnost i pouzdanost sistema.

Radi rešavanja ovih problema razvijeni su različiti mehanizmi kontrole konkurentnosti. Tradicionalni pristupi zasnovani na zaključavanju resursa omogućavaju očuvanje konzistentnosti, ali mogu dovesti do blokiranja transakcija i smanjenja performansi sistema. Kao alternativa razvijen je Multi-Version Concurrency Control (MVCC), pristup koji omogućava istovremeno izvršavanje velikog broja operacija čitanja i pisanja bez međusobnog blokiranja. Osnovna ideja MVCC-a jeste čuvanje više verzija podataka, čime svaka transakcija dobija konzistentan pogled na bazu podataka u trenutku svog pokretanja .

Predmet ovog seminarskog rada jeste analiza MVCC mehanizma u sistemu PostgreSQL, jednom od najrasprostranjenijih sistema za upravljanje relacionim bazama podataka otvorenog koda. Cilj rada je da prikaže teorijske osnove višestrukog verzionisanja, način na koji je ovaj koncept implementiran u PostgreSQL-u, kao i njegove prednosti, ograničenja i praktične implikacije u okruženjima sa velikim stepenom konkurentnosti. Posebna pažnja posvećena je internim mehanizmima za upravljanje vidljivošću podataka, izolacionim nivoima transakcija i poredjenju MVCC pristupa sa alternativnim metodama kontrole konkurentnosti.

2. MVCC koncept

Klasični pristup zasnovan na zaključavanju, posebno dvostepeno zaključavanje (2PL), sprečava anomalije konkurentnog pristupa, ali nameće ograničenja u vidu blokiranja i stanja zastoja. MVCC predstavlja alternativnu paradigmu: višestruko verzionisanje (Multi-Version Concurrency Control - MVCC). Umesto da transakcije međusobno blokiraju pristup jedinstvenoj verziji podatka, MVCC omogućava da više verzija istog logičkog sloga koegzistira, pri čemu svaka transakcija vidi onu verziju koja odgovara njenom vremenskom kontekstu. Ovaj pristup ne zamenjuje potrebu za razrešavanjem konflikata pri pisanju, ali fundamentalno menja odnos između operacija čitanja i pisanja u odnosu na klasične mehanizme zaključavanja.

2.1 Definicija i osnovni koncept

Višestruko verzionisanje (MVCC) je paradigma upravljanja konkurentnošću u kojoj sistem umesto jedne aktuelne verzije svakog podatka čuva više simultanih verzija. Svaka transakcija opaža onu verziju koja odgovara konzistentnom stanju baze podataka u okviru njenog vremenskog konteksta - snimka. Umesto da pisac zaključava resurs i time blokira čitaoce, sistem dozvoljava čitaocu da vidi prethodnu verziju podatka dok pisac gradi novu.

Fundamentalna razlika u odnosu na klasično zaključavanje može se sažeti ovako: tradicionalni pristup održava **jednu verziju + mehanizam zaključavanja**, dok MVCC održava **više verzija + pravila vidljivosti** (visibility rules). Zaključavanje sprečava pristup resursu; MVCC dozvoljava pristup, ali određuje **koju verziju** resursa transakcija sme da vidi.

Aspekt	Klasično zaključavanje (2PL)			MVCC
Čitanje	Zahteva	deljeno	(read)	Ne zahteva zaključavanje; čita vidljivu verziju iz snimka
Pisanje	Zahteva	ekskluzivno	(write)	Kreira novu verziju; stara ostaje vidljiva drugim transakcijama
Blokiranje čitalac–pisac	Uobičajeno u oba smera			Eliminiše se za par čitalac–pisac
Konflikt pisanju	pri	Rešava se	čekanjem na	Rešava se zaključavanjem, abortom ili ponavljanjem transakcije
Prostor	Jedna verzija po slogu			Više verzija po slogu

Formalno, MVCC se može definisati kao protokol upravljanja konkurentnošću koji operiše nad sledećim apstraktnim elementima:

- **Slog podataka** - logička jedinica podataka (npr. red u tabeli) koja može imati više verzija.

- **Verzija** - konkretna instanca sloga sa vrednostima i metapodacima o trenutku nastanka (vremenski žig ili identifikator transakcije).
- **Vremenski kontekst transakcije** - informacija o tome koje su transakcije bile aktivne, potvrđene ili poništene u relevantnom trenutku; u praksi se materializuje kroz snimak.
- **Pravila vidljivosti** - formalna logika koja određuje, za dati vremenski kontekst, koju verziju svakog sloga transakcija sme da vidi.

Važna napomena na apstraktnom nivou: moment uzimanja snimka zavisi od izolacionog nivoa transakcije. Kod nivoa Read Committed, snimak se može uzimati pri svakom pojedinačnom upitu, kod Repeatable Read i Serializable, snimak se obično fiksira na početku transakcije ili pri prvoj operaciji čitanja.

2.2 Zašto se MVCC koristi

MVCC se razvio kao odgovor na ograničenja klasičnih mehanizama zaključavanja.. Pesimistički pristup pretpostavlja da će konflikti biti česti i preventivno blokira pristup resursima. U radnim opterećenjima gde čitanje dominira nad pisanjem - tipičnim za OLTP sisteme, web aplikacije i mešovita opterećenja (mixed workload) - takvo blokiranje postaje usko grlo: čitaoci čekaju pisce, pisci čekaju čitaoce, a sistem može upasti u stanje zastoja (deadlock).

Na konceptualnom nivou, MVCC adresira sledeće probleme:

- **Prljavo čitanje** (dirty read) - transakcija ne vidi nepotvrđene izmene drugih transakcija, jer su ti efekti ili nevidljivi u njenom snimku, ili pripadaju verzijama koje pravila vidljivosti isključuju.
- **Neponovljivo čitanje** (non-repeatable read) - zavisi od izolacionog nivoa; snimak fiksiran na početku transakcije sprečava ovu anomaliju, dok snimak po upitu (Read Committed) je ne mora sprečiti.
- **Fantomska čitanja** (phantom read) - novi slogovi koje dodaju konkurentne transakcije mogu biti nevidljivi transakciji sa fiksiranim snimkom; stepen zaštite zavisi od izolacionog nivoa i implementacije.

Ključna prednost MVCC-a na apstraktnom nivou jeste **izolovanost čitanja od pisanja bez međusobnog blokiranja**: čitalac uvek može pronaći vidljivu verziju sloga, čak i dok pisac kreira novu. Pisac ne mora čekati da čitaoci završe, jer oni čitaju staru verziju, a ne resurs koji pisac trenutno menja.

Međutim, MVCC **ne eliminiše sve konflikte**. Kada dve transakcije pokušavaju da **istovremeno menjaju** isti slog (write–write konflikt), sistem mora primeniti dodatni mehanizam: zaključavanje na nivou sloga, detekciju konflikta pri potvrdi, abort transakcije ili ponavljanje (retry). Tipičan obrazac u tom slučaju jeste **izgubljena izmena** (lost update): obe transakcije pročitaju istu vrednost, svaka nezavisno izračuna novu, a kasnija potvrđena izmena prepisuje efekat ranije - MVCC sam po sebi to ne sprečava. MVCC rešava read–write interferenciju, ali write–write konflikt i lost update ostaju problemi koje svaka implementacija rešava dodatnim mehanizmima.

Tipični scenariji primene MVCC-a uključuju sisteme sa visokim udelom operacija čitanja, aplikacije koje zahtevaju konzistentan pogled na podatke tokom duže transakcije (analitički upiti uz paralelne operativne izmene), te okruženja gde je smanjenje latencije usled blokiranja prioritet.

2.3 Tipovi MVCC pristupa

Iako svi MVCC sistemi dele osnovnu ideju višestrukih verzija, implementacije se razlikuju po načinu na koji dodeljuju vremenski kontekst i primenjuju pravila vidljivosti. U literaturi i praksi prepoznaju se četiri dominantna tipa:

Timestamp-Based MVCC - svakoj transakciji se pri pokretanju dodeljuje vremenski žig (timestamp). Verzije slogova se porede po vremenu nastanka, a pravila čitanja i pisanja zasnivaju se na poređenju vremenskih žigova transakcije i verzije. Klasičan teorijski okvir ovog tipa jeste protokol vremenskog uređenja (timestamp ordering).

Snapshot-Based MVCC - transakcija dobija konzistentan snimak baze podataka u trenutku pokretanja transakcije ili izvršavanja upita. Snimak kodira koje su transakcije bile aktivne u tom trenutku, a vidljivost verzija se određuje u odnosu na taj snimak, a ne u odnosu na apsolutni vremenski žig.

History-Based MVCC - sistem održava punu istoriju verzija slogova, što omogućava ne samo izolaciju transakcija, već i audit, temporelne upite (temporal queries) ili vraćanje na prethodno stanje. Ovaj pristup nosi najveće zahteve u pogledu prostora i mehanizama arhiviranja.

Hybrid MVCC - kombinacija prethodnih pristupa. Tipičan primer jeste snapshot-based izolacija čitanja uz optimističku verifikaciju (Optimistic Concurrency Control - OCC) pri potvrdi transakcije, kako bi se garantovala serijalizabilnost.

Tip	Mehanizam vremenskog konteksta	Prednost	Primer sistema
Timestamp-Based	Vremenski transakcije i verzije	Žig Jednostavna formalna semantika; pogodan za teorijsku analizu	Rani istraživački sistemi
Snapshot-Based	Snimak aktivnih/potvrđenih transakcija	Visoka propusnost čitanja; prirodan za izolacione nivoe	PostgreSQL, Oracle
History-Based	Puna verziona istorija	Audit, temporelne analize	Specijalizovani temporalni DBMS
Hybrid	Snapshot + verifikacija pri commit-u	Jače konzistentnosti (serijalizabilnost)	garancije PostgreSQL SSI

2.4 Osnovni principi rada

Srž MVCC pristupa počiva na četiri međusobno povezana principa. Zajedno oni objašnjavaju kako sistem ostvaruje izolaciju bez klasičnog read–write blokiranja, ali i zašto je neophodno periodično održavanje verzionog skladišta.

Princip nedestruktivnih izmena. Kada transakcija menja određeni slog, sistem ne briše staru verziju niti je prepisuje na mestu. Umesto toga, kreira se nova verzija sa ažuriranim vrednostima, dok stara ostaje sačuvana u sistemu. U svakom trenutku mogu postojati više fizičkih reprezentacija istog logičkog sloga - po jedna za svaku verziju nastalu izmenom.

Primer: neka transakcija T1 pročitati stanje računa 10000. U međuvremenu transakcija T2 menja stanje na 9000 kreiranjem nove verzije. T1 i dalje vidi vrednost 10000, jer joj je vidljiva stara verzija; T2 vidi novu verziju nakon svoje izmene.

Princip snimaka (snapshot principle). Svaka transakcija, u trenutku pokretanja ili izvršavanja pojedinačnog upita (zavisno od izolacionog nivoa), dobija konzistentnu sliku baze podataka - snimak. Snimak određuje koje verzije pojedinih slogova su vidljive toj transakciji. Transakcija ne vidi efekte konkurentnih transakcija koje nisu bile potvrđene u trenutku relevantnog za snimak, niti efekte transakcija koje su započele nakon tog trenutka.

Primer: transakcija sa snimkom u trenutku t_0 ne vidi red umetnut od strane transakcije koja je potvrdila izmenu u trenutku $t_1 > t_0$, ako je snimak fiksiran na t_0 .

Princip izolovanosti čitanja. Operacije čitanja ne smeju blokirati operacije pisanja i obratno. Čitalac uvek može naći vidljivu verziju sloga, čak i ako pisac trenutno kreira novu verziju. Time se eliminišu situacije u kojima čitalac mora čekati da pisac oslobodi zaključavanje - najčešća i najskuplja kategorija blokiranja u sistemima zasnovanim na 2PL.

Primer: deset istovremenih upita za izveštaj može paralelno čitati istu tabelu dok jedna transakcija vrši masovno ažuriranje, bez međusobnog čekanja. Write–write konflikti i dalje zahtevaju koordinaciju, ali read–write put je potpuno nezavisan.

Princip pometanja (garbage collection principle). Budući da sistem akumulira stare verzije slogova, neophodno je periodično uklanjati one verzije koje više nijedna aktivna transakcija ne može da vidi.

Sledeći dijagram ilustruje interakciju čitaoca i pisaoca pod MVCC paradigmatom:

2.5 Timestamp ordering protokol (formalni model)

Pored opisne paradigme višestrukih verzija, MVCC može se formalizovati kroz protokol vremenskog uređenja (timestamp ordering) - jedan od klasičnih teorijskih modela iz literature o kontroli konkurentnosti. Iako ga savremeni produkcijski sistemi retko implementiraju doslovno, razumevanje ovog modela pomaže u preciznom definisanju pravila vidljivosti i konflikata.

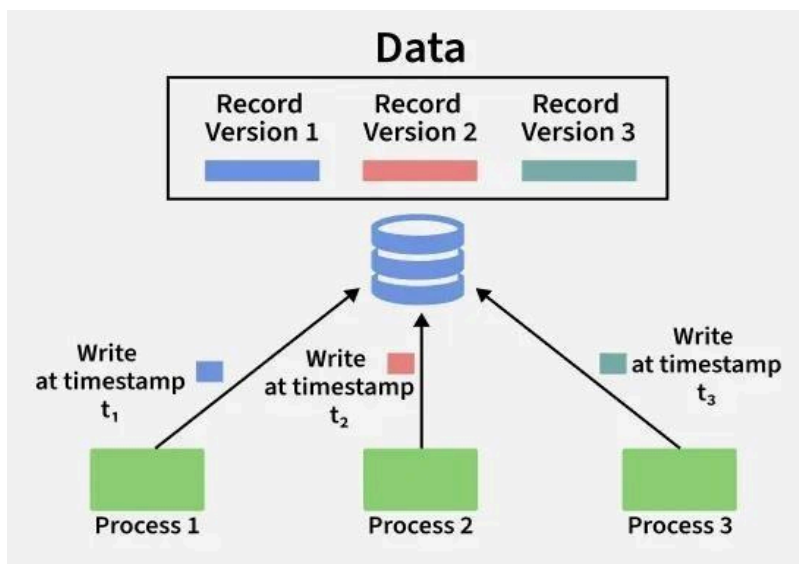
Svakoj transakciji T_i dodeljuje se vremenski žig $TS(T_i)$ pri pokretanju. Svaka verzija sloga P nosi dva vremenska žiga:

- **RTS(P)** - Read Timestamp: najveći vremenski žig transakcije koja je pročitala verziju P .
- **WTS(P)** - Write Timestamp: vremenski žig transakcije koja je kreirala verziju P .

Pravilo čitanja: Transakcija T_i sme da pročita verziju P samo ako je $WTS(P) \leq TS(T_i)$. Intuitivno, transakcija vidi samo verzije koje su nastale pre njenog vremenskog trenutka. Transakcija bira **najnoviju** verziju sloga koja zadovoljava ovaj uslov.

Pravilo pisanja: Transakcija T_i sme da kreira novu verziju P samo ako je $TS(T_i) \geq RTS(P)$. Ako je $TS(T_i) < RTS(P)$, neka transakcija sa ranijim vremenskim žigom već je pročitala staru verziju, pa bi pisanje narušilo serijalizabilnost - transakcija T_i se **poništava** (abort). U suprotnom, kreira se nova verzija sa $WTS = TS(T_i)$.

Ova dva pravila garantuju serijalizabilnost u teorijskom smislu, ali u praksi nose problem **kaskadnih poništavanja** (cascading aborts) i ne efikasno skaliraju pod visokim opterećenjem. Industrijske implementacije koriste snapshot-based pristup sa identifikatorima transakcija umesto apsolutnih vremenskih žigova, uz drugačiju logiku detekcije konflikata.



MVCC kod baza podataka

3. PostgreSQL kao sistem za upravljanje bazama podataka

PostgreSQL je napredni sistem za upravljanje relacionim bazama podataka otvorenog koda. Sistem se odlikuje visokim stepenom usaglašenosti sa SQL standardima, bogatim skupom naprednih tipova podataka, snažnom podrškom za proširivost i robusnim mehanizmima garantovanog integriteta podataka. PostgreSQL podržava napredne funkcije poput tabela sa nasleđivanjem, korisničkih funkcija pisanih u više programskih jezika, indekse različitih tipova (B-Tree, Hash, GiST, GIN, BRIN), punu podršku za JSON i JSONB formate za rad sa polustrukturiranim podacima

4.1 Arhitektura PostgreSQL

Arhitektura PostgreSQL-a zasnovana je na modelu klijent-server, pri čemu serverska komponenta upravlja svim aspektima pristupa podacima, dok klijentska aplikacija komunicira sa serverom putem dobro definisanog protokola. Svaka klijentska konekcija odgovara jednom serverskom procesu, što je karakteristično za takozvanu process-per-connection arhitekturu. Ključne komponente PostgreSQL arhitekture uključuju:

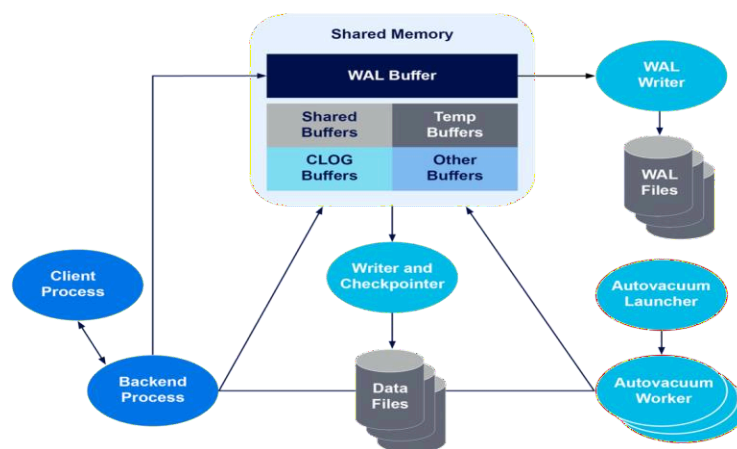
Shared Buffer Cache - zajednički bafer u memoriji koji sadrži stranice podataka i indeksa. Sistem nastoji da što više pristupa apsorbira direktno iz memorije, minimizirajući skupo čitanje sa diska.

Write-Ahead Log (WAL) - dnevnik transakcija koji snima svaku izmenu pre nego što ona bude primenjena na stvarne stranice podataka. WAL je osnova trajnosti transakcija i mehanizam oporavka u slučaju kvara.

Background Writer i Checkpointer - pozadinski procesi koji su odgovorni za periodično zapisivanje izmenjenih stranica iz bafera na disk.

Autovacuum Daemon - pozadinski proces koji automatski pokreće operacije pometanja i analize statistika.

Lock Manager - upravljač zaključavanjima koji koordinira zaključavanja na različitim granularitetima, od nivoa slogova do nivoa tabela.



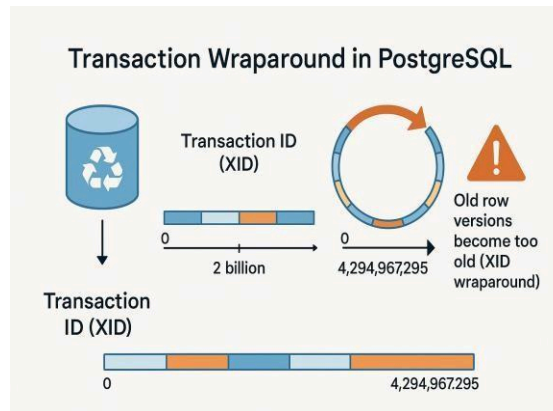
Osnove PostgreSQL arhitekture

3.2 Upravljanje transakcijama u PostgreSQL-u

PostgreSQL pridružuje svakoj transakciji jedinstveni numerički identifikator poznat kao identifikator transakcije (Transaction ID - XID). Ovaj identifikator je 32-bitni ceo broj koji se dodeljuje sekvencijalno, što znači da sistem može razlikovati starije od novijih transakcija na osnovu numeričke vrednosti identifikatora.

Svaka transakcija prolazi kroz nekoliko stanja: stanje aktivne transakcije, stanje potvrđene transakcije, stanje poništene transakcije i stanje podtransakcije. PostgreSQL čuva informacije o statusu svih skoro završenih transakcija u strukturi poznatoj kao Commit Log (CLOG), koji se koristi kao referenca pri proceni vidljivosti slogova.

Poseban izazov predstavlja takozvano prekoračenje XID-a (XID wraparound), do koga dolazi kada se iscrpi prostor 32-bitnih identifikatora transakcija. PostgreSQL rešava ovaj problem mehanizmom koji periodično zamrzava stare slogove, čime ih efektivno izvlači iz redovnog poredjenja identifikatora transakcija



Transaction Wraparound u PostgreSQL

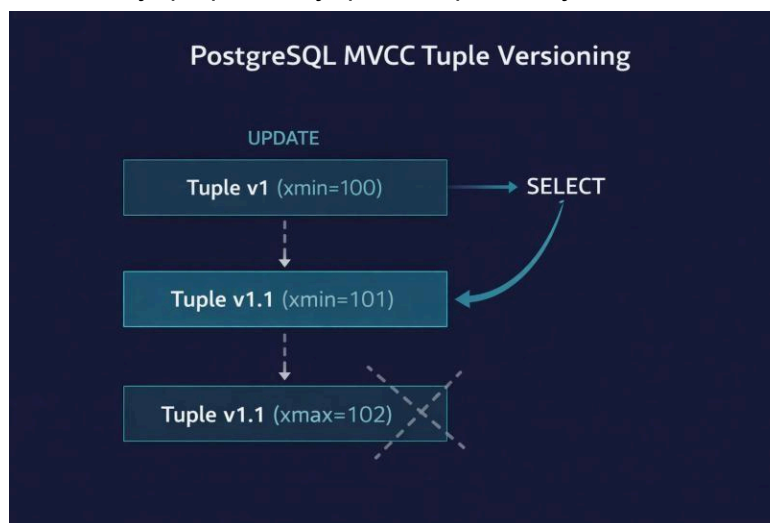
4. Implementacija MVCC mehanizma u PostgreSQL

4.1 Verzionisanje slogova (Tuple Versioning)

Temelj MVCC implementacije u PostgreSQL-u leži u konceptu verzionisanja slogova. U terminologiji PostgreSQL-a, slog se naziva tuple (čestica), i svaki logički red u tabeli može imati više fizičkih reprezentacija - jednu za svaku verziju koja je nastala kao posledica izmena .

Kada transakcija menja određeni red, PostgreSQL ne briše niti ažurira postojeći tuple. Umesto toga, stari tuple se označava kao obrisan (sa stanovišta nove verzije), a novi tuple sa ažuriranim vrednostima se upisuje kao novi fizički slog u istu tabelu. Oba tuple-a istovremeno fizički postoje u tabeli, a koja će verzija biti vidljiva određenoj transakciji određuje se pravilima vidljivosti zasnovanim na identifikatorima transakcija.

Ova strategija ima važnu arhitektonsku posledicu: tabele u PostgreSQL-u su po dizajnu append-only strukture - podaci se nikada ne menjaju na mestu (in-place update), već se uvek dodaju novi tuple-ovi. Fizičko uklanjanje zastarelih verzija prepušteno je procesu pometanja.



PostgreSQL MVCC verzionisanje slogova

4.2 Interna struktura tuple zapisa

Svaki tuple u PostgreSQL-u sadrži zaglavlje (tuple header) koje nosi metapodatke neophodne za upravljanje vidljivošću i konkurentnošću. Zaglavlje tuple-a definisano je strukturom `HeapTupleHeaderData` u izvornom kodu PostgreSQL-a i sadrži nekoliko ključnih polja :

xmin - identifikator transakcije koja je kreirala ovaj tuple (tj. koja je ubacila ili ažurirala red, kreirajući ovu verziju)

xmax - identifikator transakcije koja je obrisala ili zamenila ovaj tuple. Vrednost nula označava da tuple nije obrisana.

t_infomask i **t_infomask2** - bitmaska zastavica koje nose različite statusne informacije: da li je tuple zaključan, da li su xmin/xmax vrednosti poznate kao potvrđene ili poništene, itd.

t_ctid - fizička adresa (stranica + ofset) na kojoj se nalazi najnovija verzija ovog tuple-a. Za najnoviju verziju, **t_ctid** pokazuje na sam sebe; za starije verzije, pokazuje na sledeću u lancu verzija.

t_hoff - ofset do stvarnih korisničkih podataka u okviru tuple zapisa (jer zaglavlje može biti promenljive dužine zavisno od broja kolona i nul-mape).

4.3 Uloga xmin i xmax atributa

Atributi xmin i xmax su centralni elementi MVCC mehanizma u PostgreSQL-u. Zajedno definišu vidljivi raspon svakog tuple-a - vremenski interval, izražen u terminima identifikatora transakcija, unutar kojeg je određena verzija sloga konzistentno vidljiva transakcijama .

xmin nosi identifikator transakcije koja je kreirala tuple. Tuple je vidljiv samo onim transakcijama koje su pokrenute nakon što je transakcija xmin uspešno potvrđena (committed). Ako je transakcija xmin još uvek aktivna ili je poništena, tuple nije vidljiv nijednoj drugoj transakciji .

xmax nosi identifikator transakcije koja je obrisala ili zamenila tuple novom verzijom. Vrednost nula u xmax polju znači da tuple još uvek nije obrisana i da je potencijalno vidljiv. Kada transakcija briše red ili ažurira red (što u MVCC terminologiji znači: briše stari tuple i kreira novi), u xmax polje starog tuple-a upisuje sopstveni identifikator. Ako je transakcija sa tim identifikatorom uspešno potvrđena, tuple se smatra obrisanim sa stanovišta svih kasnijih transakcija .

Precizni algoritam vidljivosti kombinuje oba atributa: tuple je vidljiv transakciji T ukoliko je xmin potvrđena transakcija čiji je identifikator manji od identifikatora transakcije T (ili tačno jednak, ako je T sama kreirala tuple), I ukoliko je xmax ili nula, ili je aktivna/poništena transakcija, ili je potvrđena transakcija čiji je identifikator veći od identifikatora T .

4.4 Kreiranje novih verzija podataka

Proces kreiranja novih verzija podataka u PostgreSQL-u detaljno ilustruje kako MVCC funkcioniše u praksi. Razmatraju se tri osnovna scenarija: umetanje, izmena i brisanje redova.

Umetanja novog reda - PostgreSQL kreira jedan tuple čiji je xmin postavljen na identifikator tekuće transakcije, a xmax je nula. Ovaj tuple je vidljiv samo onim transakcijama koje se pokrenu nakon što transakcija umetanja bude potvrđena.

Izmene reda - PostgreSQL izvršava atomičnu operaciju koja se sastoji od dva koraka: u xmax polje postojećeg tuple-a upisuje se identifikator tekuće transakcije (čime se stara verzija označava kao obrisana), a zatim se kreira novi tuple sa ažuriranim vrednostima i xmin postavljenim na identifikator tekuće transakcije. Novi tuple u polju t_ctid starog tuple-a naznačen je kao naslednik, formirajući na taj način lanac verzija koji sistem može pratiti.

Brisanja reda - postoji samo prvi korak: u xmax polje tuple-a upisuje se identifikator tekuće transakcije. Fizičko uklanjanje tuple-a ne vrši se odmah - to je posao procesa pometanja.

4.5 Snapshot mehanizam

Snimak (snapshot) je ključni koncept koji obezbeđuje konzistentnost pogleda na podatke unutar transakcije. U PostgreSQL-u, snimak je struktura podataka koja u sebi kodira informaciju o tome koji identifikatori transakcija su bili aktivni u trenutku kreiranja snimka, koji su bili potvrđeni, a koji poništeni.

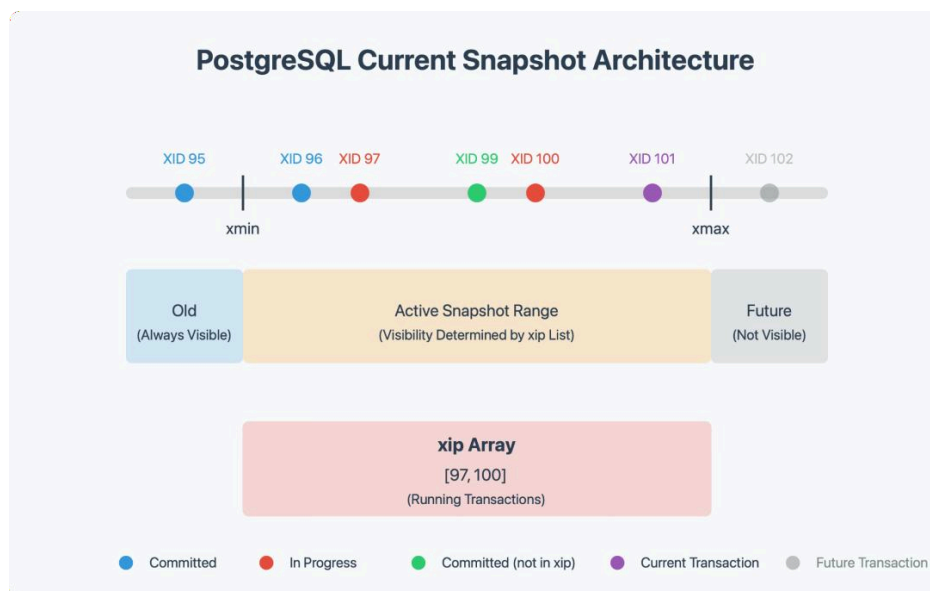
Formalno, PostgreSQL snimak sadrži tri ključna elementa:

xmin (snimka, ne tuple-a) - najmanji identifikator transakcije koji je bio aktivan u trenutku kreiranja snimka. Sve transakcije sa identifikatorom manjim od ovog xmin-a su svakako završene (potvrđene ili poništene) i njihovi efekti su poznati.

xmax (snimka) - identifikator transakcije neposredno iznad poslednje dodeljene transakcije u trenutku kreiranja snimka. Sve transakcije sa identifikatorom jednakim ili većim od ovog xmax-a nisu ni počele u vreme kreiranja snimka i stoga su nevidljive.

xip_list - lista identifikatora transakcija koje su bile aktivne (ni potvrđene ni poništene) u trenutku kreiranja snimka. Ove transakcije zahtevaju eksplicitnu proveru: budući da je njihov identifikator numerički između xmin i xmax snimka, opšta pravila poredjenja ne bi ih automatski isključila - stoga se moraju navesti eksplicitno kako bi se garantovalo da njihovi još-nepotvrđeni efekti ostanu nevidljivi.

Uz pomoć ova tri elementa, svaka transakcija može precizno odrediti koji tuple-ovi su joj vidljivi, bez potrebe za zaključavanjem.



PostgreSQL snapshot arhitektura

4.6 Obrada konkurentnih transakcija

Kada više transakcija istovremeno čita i piše u istu tabelu, MVCC mehanizam osigurava da njihove aktivnosti ne interferiraju jedna sa drugom na način koji bi narušio konzistentnost.

U tipičnom scenariju konkurentnog pristupa, transakcija T1 čita određeni skup redova, dok transakcija T2 istovremeno menja iste redove. U sistemu zasnovanom na zaključavanju, T1 bi morala čekati dok T2 ne otpusti zaključavanje. U PostgreSQL MVCC sistemu, T1 jednostavno vidi verzije tuple-ova koje su bile vidljive u trenutku kreiranja njenog snimka - verzije pre izmena koje vrši T2. T2, s druge strane, kreira nove tuple-ove koji su nevidljivi T1. Obe transakcije napreduju paralelno, bez blokiranja .

Situacija postaje složenija kada dve transakcije pokušavaju da istovremeno menjaju isti red. U tom slučaju, PostgreSQL mora koristiti zaključavanje na nivou tuple-a da bi razrešio konflikt. Transakcija koja prva stekne pravo na izmenu nastavlja sa radom; druga transakcija mora čekati da prva završi. Ako prva potvrdi izmenu, PostgreSQL mora odlučiti šta da uradi sa drugom transakcijom - zavisno od izolacionog nivoa, ona ili ponavlja svoju operaciju na novoj verziji, ili se poništava .

4.7 Upravljanje konfliktima izmedju transakcija

Konflikti izmedju transakcija u PostgreSQL MVCC sistemu rešavaju se kombinacijom snimaka i selektivnih zaključavanja. PostgreSQL koristi nekoliko vrsta zaključavanja na nivou redova: deljeno zaključavanje za čitanje (zapravo retko korišćeno zahvaljujući MVCC), zaključavanje za izmenu, i ekskluzivno zaključavanje za brisanje ili direktno zaključavanje .

Mehanizam detekcije konflikata u PostgreSQL-u, posebno na Serializable izolacionom nivou, zasnovan je na tehnici Serializable Snapshot Isolation (SSI). Ovaj mehanizam detektuje potencijalne serijalizacione konflikte praćenjem zavisnosti izmedju transakcija (read-write zavisnosti) i poništava transakciju koja bi, u slučaju potvrde, dovela do neserijalizabilnog rasporeda izvršavanja . Ova tehnika je implementirana u PostgreSQL-u od verzije 9.1 i smatra se jednim od najsofisticiranijih mehanizama kontrole konkurentnosti u industrijskim sistemima za upravljanje bazama podataka.

5. Izolacioni nivoi i MVC

5.1 Read Committed

Read Committed je podrazumevani izolacioni nivo u PostgreSQL-u i najčešće korišćen u praksi. Na ovom nivou, svaki pojedinačni upit unutar transakcije dobija novi snimak u trenutku svog izvršavanja. To znači da su promene potvrđene od strane drugih transakcija vidljive čim su potvrđene, i to u narednom upitu unutar iste transakcije .

Ovo ponašanje ima specifičnu implikaciju: unutar iste transakcije, dva uzastopna identična upita mogu dati različite rezultate, jer izmedju njih neka druga transakcija mogla potvrdi izmene. Dakle, Read Committed ne sprečava anomaliju neponovljivog čitanja (non-repeatable read), ali sprečava prljavo čitanje (dirty read), jer su vidljive samo potvrđene promene.

Na Read Committed nivou, PostgreSQL primenjuje zaključavanje samo pri upisu, a ne pri čitanju. Čitaoci nikada ne čekaju na pisce i obratno - pisci ne čekaju na čitaoce - što čini ovaj nivo izolacije posebno pogodnim za sisteme sa visokim stepenom konkurentnog čitanja .

5.2 Repeatable Read

Na Repeatable Read izolacionom nivou, PostgreSQL kreira snimak jedanput - na početku prve operacije u transakciji - i koristi ga za sve upite unutar iste transakcije. Ovo garantuje da će isti upit, izvršen dvaput unutar iste transakcije, uvek dati iste rezultate, jer oba izvršavanja koriste isti snimak .

Zanimljivo je da PostgreSQL-ova implementacija Repeatable Read de facto sprečava i fantomska čitanja (phantom reads), premda SQL standard ne zahteva ovo ponašanje na ovom nivou izolacije. Budući da se koristi isti snimak tokom cele transakcije, novi redovi koje dodaju konkurentne transakcije nisu vidljivi, čak ni ako zadovoljavaju uslov upita .

Ograničenje Repeatable Read nivoa u PostgreSQL-u je to što će transakcija, koja pokušava da izmeni red koji je u međuvremenu izmenila i potvrdila neka druga transakcija, biti automatski poništena (serialization failure). Aplikacija mora biti projektovana da obradi ovu mogućnost i eventualno ponovi transakciju .

5.3 Serializable Snapshot Isolation

Serializable izolacioni nivo u PostgreSQL-u implementiran je tehnikom Serializable Snapshot Isolation (SSI), razvijenom u akademskim istraživanjima i uvedenom u PostgreSQL od verzije 9.1 . SSI garantuje da će efekti potvrđenih transakcija biti ekvivalentni nekom serijskom rasporedu njihovog izvršavanja - kao da su izvršavane jedna za drugom, a ne paralelno.

SSI postiže ovo garantovanjem praćenjem potencijalnih cikličnih zavisnosti izmedju transakcija. Dva tipa zavisnosti su ključna: rw-zavisnost (jedna transakcija čita podatke koje menja druga) i ww-zavisnost (dve transakcije menjaju iste podatke). Kada sistem detektuje ciklus ovih zavisnosti koji bi vodio ka anomaliji, poništava jednu od transakcija u ciklusu .

SSI pristup je revolucionaran jer postiže pravi serializable nivo izolacije sa dramatično manjim brojem blokiranja u poredjenju sa klasičnim pristupima zasnovanim na 2PL. Medjutim, nosi odredjene troškove: sistem mora pratiti zavisnosti u memoriji, što povećava potrošnju resursa, a broj lažnih poništavanja (transakcija koje su poništene preventivno, a ne bi realno dovele do anomalije) može biti nenemarljiv u određenim radnim opterećenjima

5.4 Poredjenje izolacionih nivoa

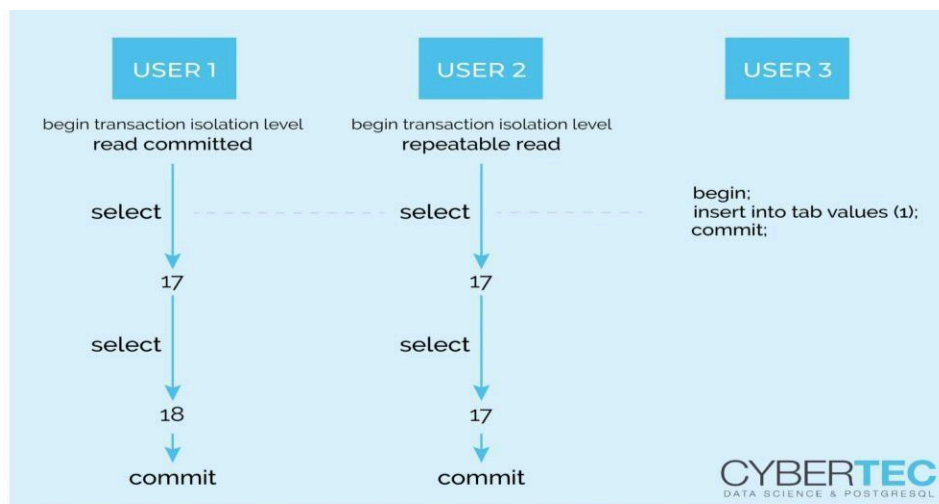
Sledeća tabela sumira anomalije koje svaki izolacioni nivo sprečava u PostgreSQL-u:

Izolacioni nivo **Dirty Read** **Non-Repeatable Read** **Phantom Read** **Serialization Anomaly**

Read Committed	Sprečeno	Moguće	Moguće	Moguće
Repeatable Read	Sprečeno	Sprečeno	Sprečeno*	Moguće
Serializable	Sprečeno	Sprečeno	Sprečeno	Sprečeno

Tabela poredjenja izolacionih nivou u PostgreSQL-u

PostgreSQL implementacija Repeatable Read sprečava phantom reads, premda SQL standard to ne zahteva na ovom nivou.



Poredjenje read committed i repeatable read

6. Upravljanje prostorom i održavanje baze

Neposredna posledica MVCC filozofije nedestruktivnih izmena jeste akumulacija zastarelih verzija tuple-ova (dead tuples) u tabelama. Svaka izmena ili brisanje reda rezultuje nastankom jednog ili više tuple-ova koji su vidljivi samo onim transakcijama koje su aktivne u trenutku izmene, a koji su nevidljivi svim kasnijim transakcijama.

Kada nijedna aktivna transakcija više ne može da vidi određeni tuple - jer su sve transakcije koje su mogle da ga vide završene - taj tuple postaje zreo za pometanje (reclaimable). Međutim, PostgreSQL ne briše ove tuple-ove automatski u trenutku kada postaju zastareli; to bi zahtevalo skupu sinhronizaciju i moglo bi negativno uticati na performanse. Umesto toga, PostgreSQL koristi poseban pozadinski mehanizam - VACUUM - za asinhronog pometanje zastarelih tuple-ova.

6.1 Data Bloat fenomen

Kada se zastareli tuple-ovi akumuliraju brže nego što se čiste, nastaje fenomen poznat kao naduvavanje podataka. Tabele i indeksi postaju fizički veći nego što bi morali biti na osnovu stvarnog broja živih redova, što ima višestruke negativne posledice .

Prvo, povećava se vreme skeniranja tabela (sequential scan) jer sistem mora da prođe kroz fizički veći broj stranica da bi pronašao iste žive tuple-ove. Drugo, povećava se potrošnja prostora u keš baferima, jer se u keš učitavaju i stranice sa visokim udelom mrtvih tuple-ova, efikasno smanjujući korisni kapacitet keša. Treće, indeksi postaju fragmentovani i sadrže pokazivače na mrtve tuple-ove, što usporava operacije pretraživanja indeksa .

Data bloat je posebno problematičan u tabelama koje su podvrgnute intenzivnim update operacijama, kao i u tabelama sa dugotrajnim transakcijama koje sprečavaju pravovremeno pometanje.

6.2 VACUUM mehanizam

VACUUM je primarni mehanizam PostgreSQL-a za upravljanje zastarelim tuple-ovima i suzbijanje data bloat-a. U osnovi, VACUUM prolazi kroz stranice tabele, identifikuje mrtve tuple-ove i oslobadja prostor koji oni zauzimaju za ponovnu upotrebu od strane budućih operacija umetanja ili ažuriranja .

Važno je napomenuti da standardni VACUUM ne vraća prostor operativnom sistemu - on samo označi oslobodjene stranice kao dostupne za buduću upotrebu unutar PostgreSQL-ovog sopstvenog upravljanja prostorom. Izuzetak su određene situacije u kojima VACUUM može fizički ukloniti prazne stranice s kraja heap fajla i vratiti ih operativnom sistemu, ali ovo je ograničeno samo na stranice na kraju fajla. Fizičko smanjivanje celokupne datoteke tabele zahteva invazivniju operaciju (VACUUM FULL) .

Pored upravljanja mrtvim tuple-ovima, VACUUM obavlja još jednu kritičnu funkciju: ažurira statistike vidljivosti stranica u Visibility Map - bitmapi koja prati koje stranice sadrže isključivo žive tuple-ove vidljive svim transakcijama. Ova informacija koristi se za optimizaciju, jer stranice markirane u Visibility Map mogu biti preskočene tokom Index-Only Scan operacija .

Treća funkcija VACUUM-a je zamrzavanje (freezing) starih tuple-ova radi rešavanja problematike prekoračenja XID-a. Tuple-ovi čiji je xmin dovoljno star zamrzavaju se dodavanjem posebne zastavice, čime se praktično izvlače iz normalnog poredjenja identifikatora transakcija .

6.3 Autovacuum

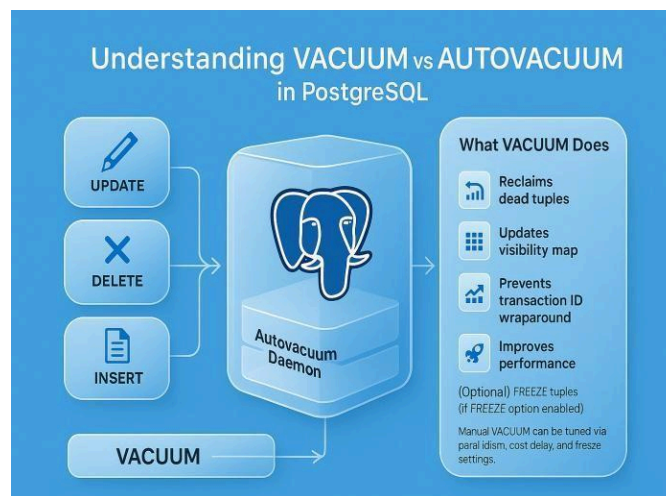
Autovacuum je mehanizam koji automatski pokreće VACUUM i ANALYZE operacije kada je to potrebno, eliminišući potrebu za ručnim administriranjem. Autovacuum daemon (pozadinski proces) periodično proverava statistike o broju mrtvih tuple-ova u svakoj tabeli i pokreće VACUUM kada taj broj predje konfigurisani prag .

Ključni konfiguracioni parametri autovacuum-a uključuju:

- **autovacuum_vacuum_threshold** - minimalan broj mrtvih tuple-ova pre pokretanja VACUUM-a (podrazumevano: 50).
- **autovacuum_vacuum_scale_factor** - deo ukupnog broja redova koji se dodaje na threshold (podrazumevano: 0.2, tj. 20% tabele).

- **autovacuum_naptime** - interval između provera (podrazumevano: 1 minut).
- **autovacuum_vacuum_cost_delay** - throttling parametar koji usporava autovacuum kako bi smanjio njegov uticaj na regularne operacije.

Autovacuum je projektovan da radi tiho u pozadini uz minimalan uticaj na performanse, ali u sistemima sa izuzetno visokom stopom izmena može biti potrebno agresivno podešavanje ovih parametara



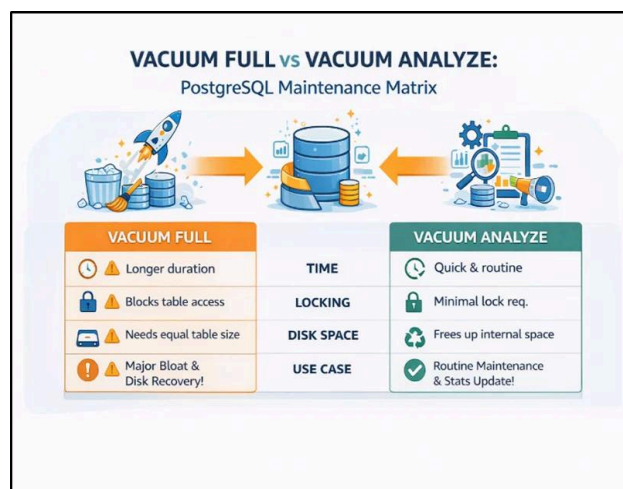
Poredjenje vacuum vs autovacuum

6.4 VACUUM FULL

VACUUM FULL je invazivnija varijanta VACUUM operacije koja fizički reorganizuje tabelu i vraća neiskorišćeni prostor operativnom sistemu. Za razliku od standardnog VACUUM-a, VACUUM FULL kreira potpuno novu kopiju tabele, prebacuje sve žive tuple-ove u novu kopiju, a zatim briše staru kopiju .

Ova operacija ima ozbiljan nedostatak: zahteva ekskluzivno zaključavanje tabele tokom celog procesa, što znači da nijedna druga transakcija ne može čitati niti pisati u tabelu dok VACUUM FULL traje. Za velike tabele, ovo može značiti više sati nedostupnosti, što je neprihvatljivo za sisteme visoke dostupnosti.

Kao alternativa VACUUM FULL-u, za minimalno invazivnu reorganizaciju tabela može se koristiti ekstenzija pg_repack, koja postiže sličan efekat bez dugotrajnog ekskluzivnog zaključavanja .



Poredjenje vacuum full vs vakuum

7. Demonstracija MVCC mehanizma u PostgreSQL

Praktični deo ovog seminarskog rada organizovan je kao niz koherentnih SQL eksperimenata koji direktno demonstriraju teorijske koncepte opisane u prethodnim poglavljima. Svaka demonstracija projektovana je tako da potvrdi konkretnu tvrdnju iz teorijskog dela: od strukture internih atributa tuple-ova, preko ponašanja izolacionih nivoa u paralelnim transakcijama, do merljivog uticaja data bloat-a na performanse sistema.

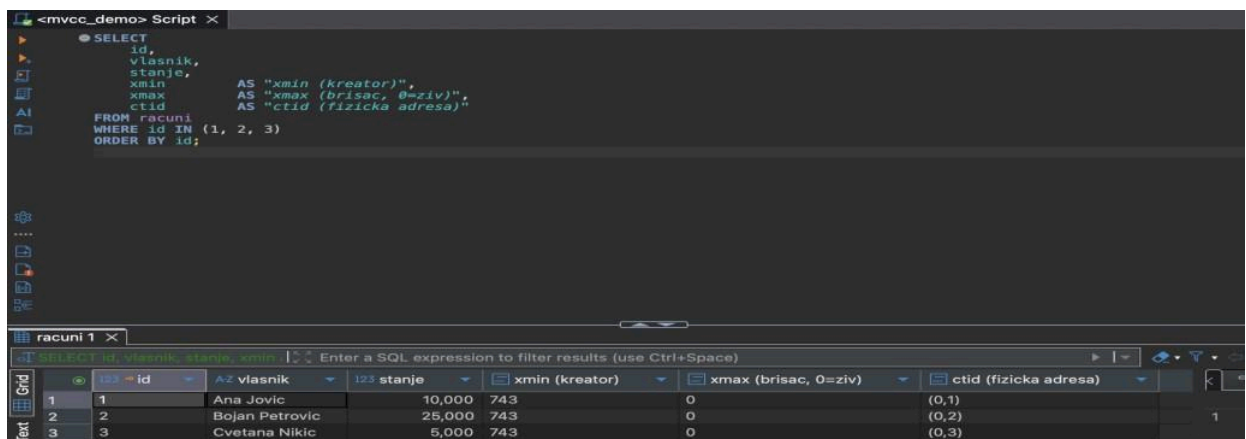
Eksperimenti su realizovani na PostgreSQL verziji 16, pokrenuto u Docker kontejneru sa standardnom konfiguracijom. Test šema obuhvata tabelu racuni (finansijski scenario sa 1005 redova) i tabelu proizvodi (katalog sa 505 redova), čime se postiže dovoljno velika baza podataka za merljive rezultate bez nepotrebnih zahteva za resursima. Instaliraju se ekstenzije pageinspect i pgstattuple, koje su deo standardne PostgreSQL distribucije i omogućavaju direktan uvid u fizičku strukturu heap stranica.

Metodološki pristup zasnovan je na principu kontrolisanih eksperimenata: svaka demonstracija počinje poznatim, resetovanim stanjem podataka, izvršavaju se tačno određene operacije, a zatim se mere i interpretiraju rezultati. Tamo gde demonstracija neophodno zahteva dve paralelne sesije - demonstracije izolacionih nivoa i anomalija, naglaseno je na primerima.

7.1 Inspekcija tuple zaglavlja i xmin/xmax atributa

Sistemske atributi tuple-a

Osnova MVCC arhitekture u PostgreSQL-u leži u zaglavlju svakog tuple-a, koje nosi metapodatke o vidljivosti. PostgreSQL izlaže interne atribute tuple-a kao skrivene sistemske kolone koje se mogu eksplicitno zatražiti u SELECT iskazu. Tri najvažnija su xmin, xmax i ctid.



```
SELECT
  id,
  vlasnik,
  stanje,
  xmin AS "xmin (kreator)",
  xmax AS "xmax (brisac, 0=ziv)",
  ctid AS "ctid (fizicka adresa)"
FROM racuni
WHERE id IN (1, 2, 3)
ORDER BY id;
```

	id	vlasnik	stanje	xmin (kreator)	xmax (brisac, 0=ziv)	ctid (fizicka adresa)
1	1	Ana Jovic	10,000	743	0	(0,1)
2	2	Bojan Petrovic	25,000	743	0	(0,2)
3	3	Cvetana Nikic	5,000	743	0	(0,3)

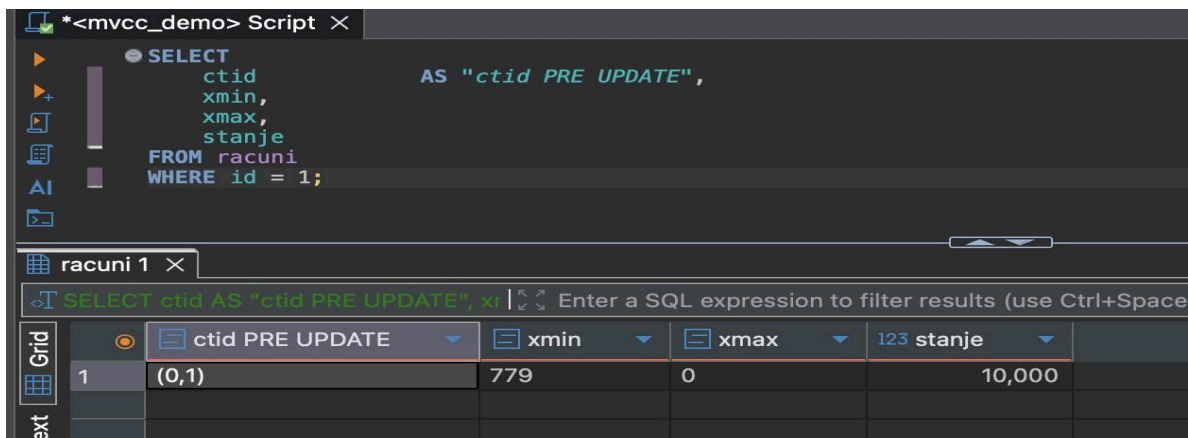
Čitanje sistemskih atributa tuple-a

Konkretna vrednost XID-a zavisi od broja prethodnih transakcija u datoj instalaciji i razlikovaće se pri svakom pokretanju.

Analiza rezultata: Svi redovi imaju xmax = 0, što prema pravilima vidljivosti znači da nijedan nije obrisan - svi su živi tuple-ovi. Kolona ctid prikazuje fizičku adresu u formatu (broj_stranice, slot): vrednost (0,1) znači da se tuple nalazi na stranici 0, u slotu 1. Identičan xmin za sve tri redova potvrđuje da su ubačeni u istoj transakciji.

UPDATE kao dvofazna operacija

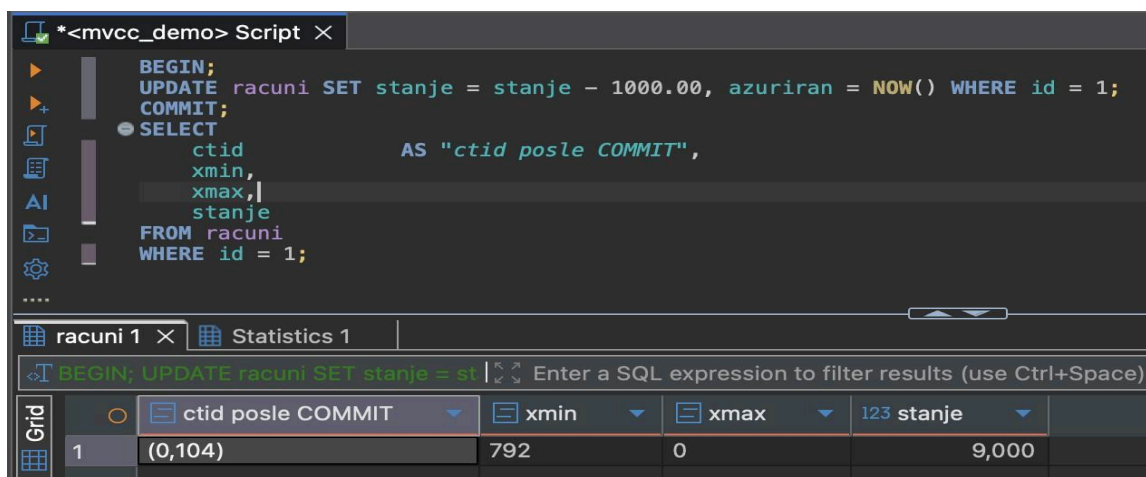
Najvažnija arhitektonska karakteristika PostgreSQL MVCC-a jeste da UPDATE nije modifikacija podataka na mestu, već kreiranje nove verzije tuple-a. Sledeći primer to direktno potvrđuje posmatranjem promene ctid i vrednosti xmin/xmax.



The screenshot shows a PostgreSQL query window titled '*<mvcc_demo> Script'. The query is a SELECT statement from the 'racuni' table where id = 1. The results grid shows one row with the following values:

	ctid PRE UPDATE	xmin	xmax	123 stanje
1	(0,1)	779	0	10,000

Stanje pre UPDATE-a



The screenshot shows a PostgreSQL query window titled '*<mvcc_demo> Script'. The query is an UPDATE statement followed by a SELECT statement. The results grid shows one row with the following values:

	ctid posle COMMIT	xmin	xmax	123 stanje
1	(0,104)	792	0	9,000

Posle UPDATE i COMMIT

Analiza rezultata: ctid se promenio iz (0,1) u (0,104), što direktno potvrđuje da je novi tuple smešten na različitu fizičku lokaciju. Novi xmin (792) odgovara XID-u UPDATE transakcije. Stari tuple sa ctid = (0,1) i xmax = 792 i dalje fizički postoji u heap-u kao mrtve tuple, vidljiv jedino kroz pageinspect, ali ne i kroz standardne upite .

Inspekcija heap stranice: pageinspect

Ekstenzija pageinspect pruža direktan uvid u fizički sadržaj heap stranica, uključujući i mrtve tuple-ove koji su nevidljivi regularnim upitima. Ovo je najdirektnije moguće eksperimentalno potvrđivanje tvrdnje da su tabele u PostgreSQL-u append-only strukture - stare verzije ostaju u heap-u dok ih VACUUM ne ukloni.

Inspekcija stranice 0 posle UPDATE-a na redu id = 1 pokazuje mrtvi tuple u slotu 1: xmin = 743, xmax = 767, xmax_status = AKTIVAN, t_ctid = (10,27). Polje t_ctid ne pokazuje na (0,7) kao u minimalnom primeru sa tri reda, već na stranicu 10, slot 27 - nova verzija tuple-a smeštena je na drugu stranicu jer je stranica 0 već bila popunjena podacima iz setup.sql (1005 redova). To ne menja MVCC logiku: stara verzija je označena u xmax, nova je kreirana odvojeno, a lanac verzija povezuje ih kroz t_ctid.

Napomena o xmax_status: vrednost AKTIVAN u ovom upitu ne znači nužno da izmena nije potvrđena. Upit čita sirovu t_infomask bitmasku na stranici; bit za potvrđeni xmax (1024) na mrtvom tuple-u u pageinspect prikazu ponekad nije postavljen, iako regularan SELECT više ne vidi tu verziju. Za demonstraciju MVCC-a ključno je da postoji mrtvi tuple sa xmax ≠ 0 i da t_ctid vodi ka novoj verziji.

```

SELECT
  lp AS slot,
  t_xmin AS xmin,
  t_xmax AS xmax,
  t_ctid AS ctid,
  CASE
    WHEN (t_infomask & 256) != 0 THEN 'POTVRDEN'
    ELSE 'AKTIVAN/PONISTEN'
  END AS xmin_status,
  CASE
    WHEN t_xmax = 0 THEN '(nema)'
    WHEN (t_infomask & 1024) != 0 THEN 'POTVRDEN'
    ELSE 'AKTIVAN'
  END AS xmax_status
FROM heap_page_items(get_raw_page('racuni', 0))
WHERE t_xmin IS NOT NULL
ORDER BY lp;

```

	slot	xmin	xmax	ctid	xmin_status	xmax_status
1	1	743	767	(10,27)	POTVRDEN	AKTIVAN
2	2	743	0	(0,2)	POTVRDEN	(nema)
3	3	743	0	(0,3)	POTVRDEN	(nema)
4	4	743	0	(0,4)	POTVRDEN	(nema)
5	5	743	0	(0,5)	POTVRDEN	(nema)
6	6	744	0	(0,6)	POTVRDEN	(nema)
7	7	744	0	(0,7)	POTVRDEN	(nema)
8	8	744	0	(0,8)	POTVRDEN	(nema)
9	9	744	0	(0,9)	POTVRDEN	(nema)
10	10	744	0	(0,10)	POTVRDEN	(nema)
11	11	744	0	(0,11)	POTVRDEN	(nema)
12	12	744	0	(0,12)	POTVRDEN	(nema)
13	13	744	0	(0,13)	POTVRDEN	(nema)
14	14	744	0	(0,14)	POTVRDEN	(nema)

Sadržaj heap stranice 0 (primer posle UPDATE-a)

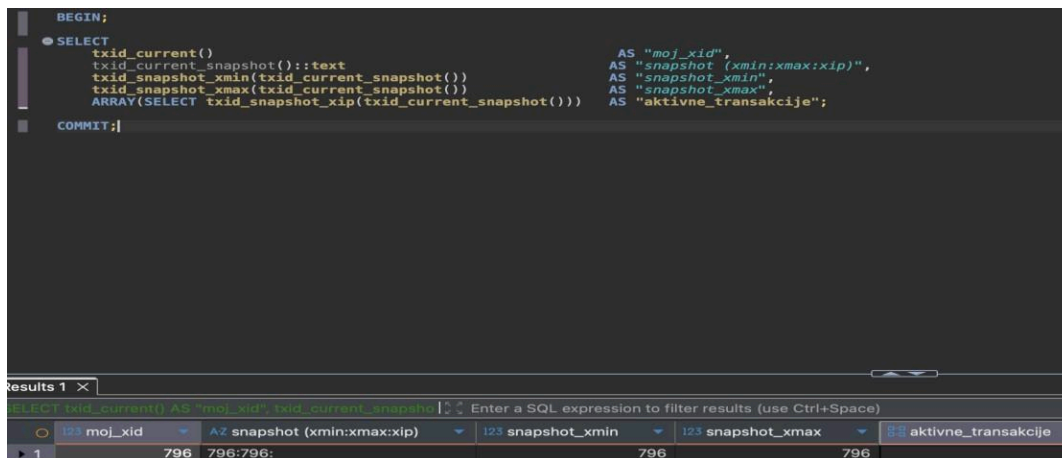
Konkretni XID-ovi (743, 767) zavise od broja prethodnih transakcija u datoj instalaciji.

Analiza rezultata: Mrtvi tuple (slot 1, xmax = 767) i nova verzija na (10,27) potvrđuju princip nedestruktivnih izmena: UPDATE nije prepisao staru verziju na mestu, već je označio staru i kreirao novu. Za screenshot u seminarskom dovoljno je prikazati slot 1 sa xmax ≠ 0 i t_ctid koji pokazuje na novu lokaciju, uz redove 2–3 kao primer živih tuple-ova sa xmax = 0.

Anatomija snimka transakcije

Format snimka je xmin:xmax:xip_lista. Dobijeni snimak 796:796: pokazuje da su xmin i xmax jednaki (oba 796), što znači da u trenutku pokretanja nije bilo nijedne druge aktivne transakcije. Prazna xip lista potvrđuje odsustvo konkurentnih transakcija.

U produkcijskom sistemu sa paralelnim transakcijama, snimak bi izgledao npr. 781:797:783,788 - što znači da su transakcije 783 i 788 aktivne i moraju biti eksplicitno isključene iz vidljivosti, uprkos tome što su im XID-ovi numerički između xmin i xmax snimka .



```
BEGIN;
SELECT
  txid_current() AS "moj_xid",
  txid_current_snapshot()::text AS "snapshot (xmin:xmax:xip)",
  txid_snapshot_xmin(txid_current_snapshot()) AS "snapshot_xmin",
  txid_snapshot_xmax(txid_current_snapshot()) AS "snapshot_xmax",
  ARRAY(SELECT txid_snapshot_xip(txid_current_snapshot())) AS "aktivne_transakcije";
COMMIT;
```

Results 1 x

	moj_xid	snapshot (xmin:xmax:xip)	snapshot_xmin	snapshot_xmax	aktivne_transakcije
1	796	796:796:	796	796	

Elementi snimka tekuće transakcije

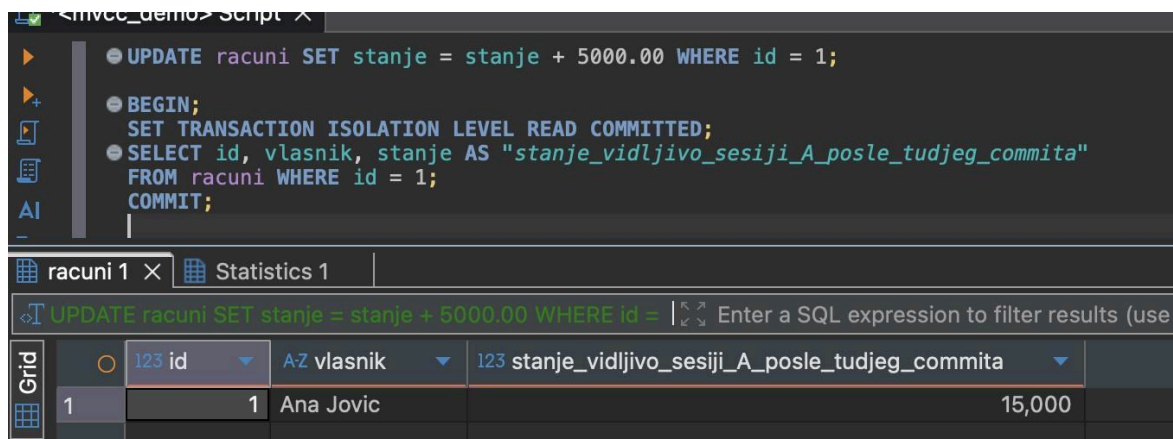
Analiza rezultata: Format snimka je xmin:xmax:xip_lista. Dobijeni snimak 796:796: pokazuje da je xmin jednak xmax-u (oba 796), što znači da u trenutku pokretanja nije bilo nijedne druge aktivne transakcije - jedina transakcija u sistemu bila je upravo tekuća (XID 796). Prazna xip lista (`` bez vrednosti iza) potvrđuje odsustvo konkurentnih transakcija koje bi morale biti eksplicitno isključene iz vidljivosti. Ovaj rezultat je tipičan za test okruženje sa jednom sesijom. U produkcijskom sistemu sa paralelnim transakcijama, snimak bi izgledao npr. `781:797:783,788` - što bi značilo da su transakcije 783 i 788 aktivne i moraju biti eksplicitno isključene iz vidljivosti, uprkos tome što su njihovi XID-ovi numerički između xmin i xmax snimka. Trostepena struktura snimka opisana ostaje relevantna bez obzira na to da li je xip lista popunjena ili prazna .

7.2 Demonstracija izolacionih nivoa u paralelnim sesijama

Read Committed: snimak po upitu

Na podrazumevanom Read Committed nivou, svaki upit u transakciji dobija novi snimak. Posledica je non-repeatable read anomalija - isti upit može dati različite rezultate unutar iste transakcije. Tabela read committed na kojoj je prikazan tok paralelnih sesija:

Vreme	Sesija A (READ COMMITTED)	Sesija B
t1	BEGIN;	-
t2	SELECT stanje FROM racuni WHERE id=1; - → 10000.00	
t3	-	BEGIN;
t4	-	UPDATE racuni SET stanje=15000 WHERE id=1;
t5	-	COMMIT;
t6	SELECT stanje FROM racuni WHERE id=1; → 15000.00	
t7	COMMIT;	-



The screenshot shows a database client interface. The top pane displays a SQL script:

```

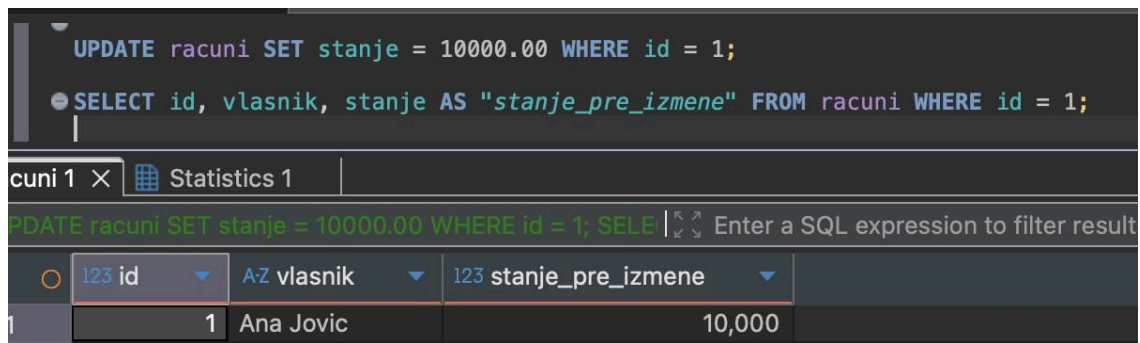
UPDATE racuni SET stanje = stanje + 5000.00 WHERE id = 1;
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT id, vlasnik, stanje AS "stanje_vidljivo_sesiji_A_posle_tudjeg_commita"
FROM racuni WHERE id = 1;
COMMIT;

```

The bottom pane shows a table named "racuni 1" with the following data:

id	vlasnik	stanje
1	Ana Jovic	15,000

Simuliramo staro stanje koje je Sesija A pročitala



The screenshot shows a database client interface. The top pane displays a SQL script:

```

UPDATE racuni SET stanje = 10000.00 WHERE id = 1;
SELECT id, vlasnik, stanje AS "stanje_pre_izmene" FROM racuni WHERE id = 1;

```

The bottom pane shows a table named "racuni 1" with the following data:

id	vlasnik	stanje
1	Ana Jovic	10,000

Sesija A na Read Committed: novi upit vidi izmenu

Analiza rezultata: Na koraku t6, Sesija A vidi vrednost 15000.00 iako je njen upit na koraku t2 video 10000.00. MVCC mehanizam dozvoljava ovo ponašanje na read committed nivou jer se novi snimak uzima pri svakom upitu, a Sesija B je u međuvremenu potvrdila izmenu. Dirty read je ipak sprečen - nepotvrdjena vrednost iz t4 nije bila vidljiva .

Repeatable Read: konzistentni snimak

Na Repeatable Read nivou, PostgreSQL kreira snimak jedanput - pri prvoj operaciji u transakciji - i koristi ga za sve naredne upite. Ovo garantuje da isti upit, izvršen dvaput unutar iste transakcije, uvek vraća iste rezultate, bez obzira na to šta su u međuvremenu uradile druge transakcije.

Tabela repeatable read: isti snimak kroz celu transakcij

Vreme	Sesija A (REPEATABLE READ)	Sesija B
t1	BEGIN; SET ISOLATION LEVEL REPEATABLE READ;	-
t2	SELECT stanje FROM racuni WHERE id=1; - → 10000.00	
t3	-	BEGIN;
t4	-	UPDATE racuni SET stanje=20000 WHERE id=1;
t5	-	COMMIT;
t6	SELECT stanje FROM racuni WHERE id=1; - → 10000.00	
t7	COMMIT;	-

```

BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

SELECT id, vlasnik, stanje AS "citanje_1 (ocekujemo 10000)"
FROM racuni WHERE id = 1;

SELECT
    txid_current()                AS "xid_sesije_A",
    txid_snapshot_xmin(txid_current_snapshot()) AS "snapshot_xmin",
    txid_snapshot_xmax(txid_current_snapshot()) AS "snapshot_xmax";

SELECT id, vlasnik, stanje AS "citanje_2"
FROM racuni WHERE id = 1;

COMMIT;

```

oba SELECT-a iz Sesije A - oba vraćaju 10000.00 uprkos commit-u Sesije B*

```

-- sesija B
BEGIN;
UPDATE racuni SET stanje = 20000.00 WHERE id = 1;
COMMIT;

```

Update iz Sesije B

Analiza rezultata: Na koraku t6, Sesija A i dalje vidi vrednost 10000.00, uprkos tome što je Sesija B na t5 potvrdila izmenu na 20000.00. PostgreSQL koristi snimak uzet na t2 za sve upite u Sesiji A. Ako bi Sesija A pokušala UPDATE na t7, PostgreSQL bi vratio grešku ERROR 40001 - serialization failure .

Serializable SSI: detekcija write skew anomalije

SSI mehanizam prati read-write zavisnosti i detektuje cikluse koji bi doveli do nesarijalizabilnog rasporeda. Write skew je anomalija u kojoj svaka transakcija čita podatke koje menja druga.

Vreme	Sesija A (SERIALIZABLE)	Sesija B (SERIALIZABLE)
t1	BEGIN; SET ISOLATION LEVEL SERIALIZABLE;	BEGIN; SET ISOLATION LEVEL SERIALIZABLE;
t2	SELECT SUM(stanje) FROM racuni WHERE id IN (1,2); → 2000	SELECT SUM(stanje) FROM racuni WHERE id IN (1,2); → 2000
t3	UPDATE racuni SET stanje = stanje - 1500 WHERE id = 1;	UPDATE racuni SET stanje = stanje - 1500 WHERE id = 2;
t4	COMMIT; → uspešno	-
t5	-	COMMIT; → ERROR 40001

Tabela SSI write skew scenario (zbir stanja ne sme biti negativan)

```

--<mvcc_demo> Script X
UPDATE racuni SET stanje = 1000.00 WHERE id = 1; |
UPDATE racuni SET stanje = 1000.00 WHERE id = 2;

BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- SELECT
-- 'SSI aktivan'
-- current_setting('transaction_isolation') AS status,
-- txid_current() AS nivo_izolacije,
-- txid_current() AS xid;

-- Prikaz SSI praćenja zavisnosti (pg_locks)
SELECT
    locktype,
    relation::regclass,
    mode,
    granted,
    pid
FROM pg_locks
WHERE pid = pg_backend_pid()
AND locktype IN ('relation', 'tuple', 'transactionid')
ORDER BY locktype;

COMMIT;

```

AZ locktype	AZ relation	AZ mode	granted	pid	
1	relation	pg_locks	AccessShareLock	[v]	34
2	transactionid	[NULL]	ExclusiveLock	[v]	34

SQL Error [23514]: ERROR: new row for relation "racuni" violates check constraint "stanje_pozitivno"
 Detail: Failing row contains (1, Ana Jovic, -500.00, tekuci, 2026-06-15 23:04:27.752264+02, 2026-06-15 23:04:27.752264+02).

Prikaz SSI praćenja zavisnosti

Analiza rezultata: Sesija B dobija grešku serializacije jer bi potvrda obe transakcije dovela do ukupnog stanja -1000, što narušava poslovni uslov. SSI je detektovao cikličnu zavisnost: Sesija A je čitala podatke koje menja Sesija B i obrnuto, formirajući ciklus koji implicira anomaliju .

7.3 Anomalije konkurentnog izvršavanja i MVCC zaštita

Dirty Read - nemoguće u PostgreSQL-u

Dirty read nastaje kada transakcija čita nepotvrdjene izmene druge transakcije. PostgreSQL sprečava ovu anomaliju na svim izolacionim nivoima, jer su u snimku uvek vidljive samo potvrđene verzije tuple-ova.

```

BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- SELECT id, vlasnik, stanje AS "vidljivo_stanje_ne_vidimo_nepotvrđenu_izmenu"
-- FROM racuni WHERE id = 1;

COMMIT;

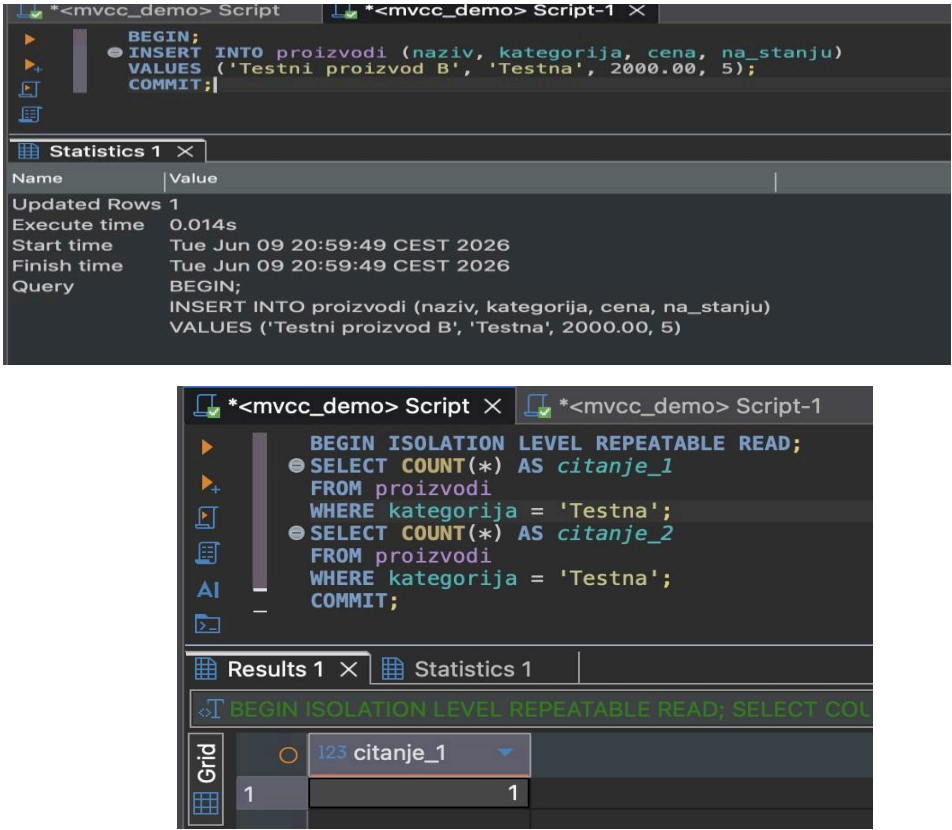
```

id	vlasnik	vidljivo_stanje_ne_vidimo_nepotvrđenu_izmenu
1	Ana Jovic	10,000

Dirty Read

Analiza rezultata: Sesija A vidi vrednost 10000.00 bez obzira na to što Sesija B ima aktivnu nepotvdjenu transakciju koja je postavila vrednost na 99999.00. MVCC mehanizam garantuje ovaj ishod: snimak koji Sesija A koristi sadrži samo potvrđene verzije, a nepotvđjena izmena Sesije B je isključena iz vidljivosti .

Phantom read sprečen na repeatable read



Phantom read demonstracija

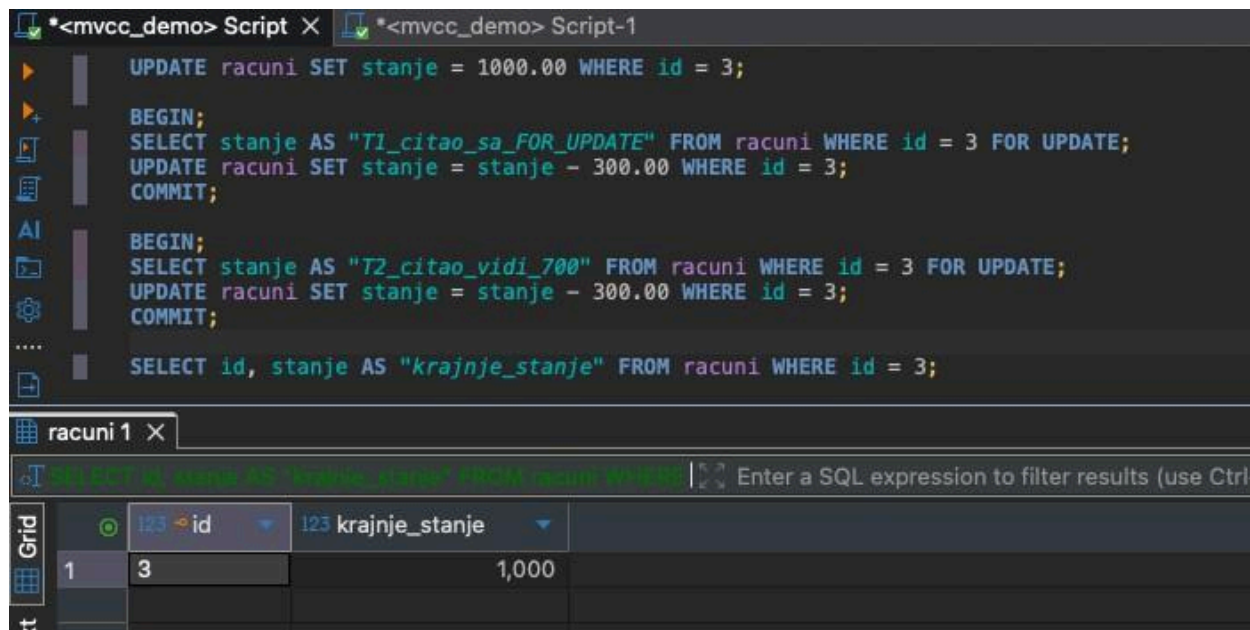
Tabela Phantom read demonstracija

Čitanje	Rezultat (bez zaštite)	Rezultat (Repeatable Read)
1.	1	1
2.	2 (phantom!)	1 (zaštićeno)

Analiza rezultata: Na Repeatable Read nivou, novi red koji je ubacila Sesija B ima xmin koji je veći od xmax snimka Sesije A (nastao je nakon uzimanja snimka), te je po pravilima vidljivosti nevidljiv Sesiji A. Ovaj rezultat je direktna posledica snapshot mehanizma .

Lost Update i SELECT FOR UPDATE

Lost update nastaje u scenarijima gde aplikacija čita vrednost, izračunava novu i upisuje rezultat bez eksplicitne zaštite - obe transakcije upisuju vrednost zasnovanu na istom pročitanoj stanju.



```
UPDATE racuni SET stanje = 1000.00 WHERE id = 3;

BEGIN;
SELECT stanje AS "T1_citao_sa_FOR_UPDATE" FROM racuni WHERE id = 3 FOR UPDATE;
UPDATE racuni SET stanje = stanje - 300.00 WHERE id = 3;
COMMIT;

BEGIN;
SELECT stanje AS "T2_citao_vidi_700" FROM racuni WHERE id = 3 FOR UPDATE;
UPDATE racuni SET stanje = stanje - 300.00 WHERE id = 3;
COMMIT;

SELECT id, stanje AS "krajnje_stanje" FROM racuni WHERE id = 3;
```

id	krajnje_stanje
3	1,000

Lost update scenario bez zaštite

Analiza rezultata: SELECT FOR UPDATE rešava lost update eksplicitnim zaključavanjem na nivou reda. Ovo je jedan od slučajeva u kojima PostgreSQL kombinuje MVCC i klasično zaključavanje: MVCC osigurava konzistentno čitanje, ali eksplicitno zaključavanje je neophodno za atomičnost sekvence čitaj-izračunaj-upiši .

Tabela rekapitulacija zaštite po izolacionim nivoima:

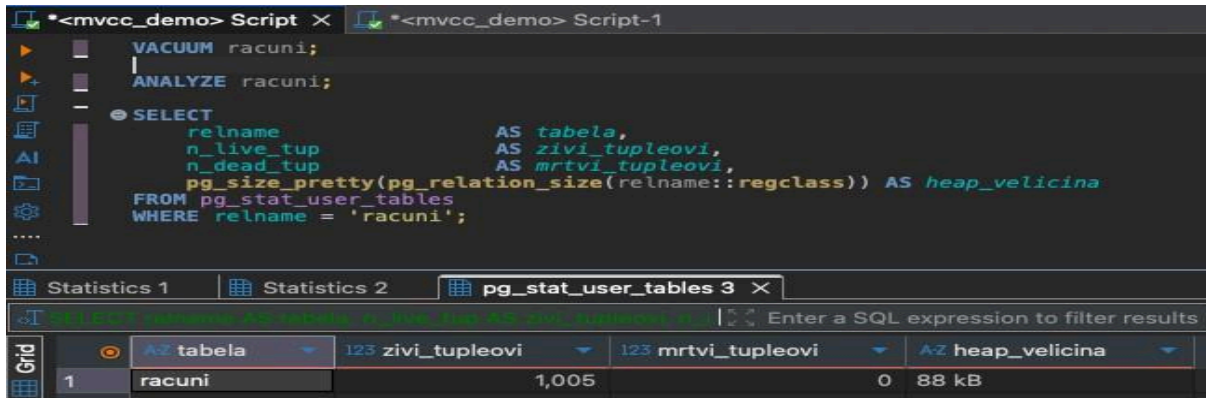
Anomalija	Read Committed	Repeatable Read	Serializable
Dirty Read	Sprečeno	Sprečeno	Sprečeno
Non-Repeatable Read	Moguće	Sprečeno	Sprečeno
Phantom Read	Moguće	Sprečeno*	Sprečeno
Lost Update	Moguće	Moguće†	Sprečeno
Serialization An.	Moguće	Moguće	Sprečeno

*PostgreSQL Repeatable Read sprečava phantom reads iako SQL standard to ne zahteva. †Na Repeatable Read, pokušaj pisanja u izmenjeni red rezultuje serialization failure greškom

7.4 Upravljanje mrtvim tuple-ovima i VACUUM analiza

Generisanje i merenje data bloat-a

Svaka izmena podataka u PostgreSQL-u generiše mrtve tuple-ove koji zauzimaju prostor. Sistemski pogled pg_stat_user_tables omogućava praćenje broja živih i mrtvih tuple-ova po tabeli.

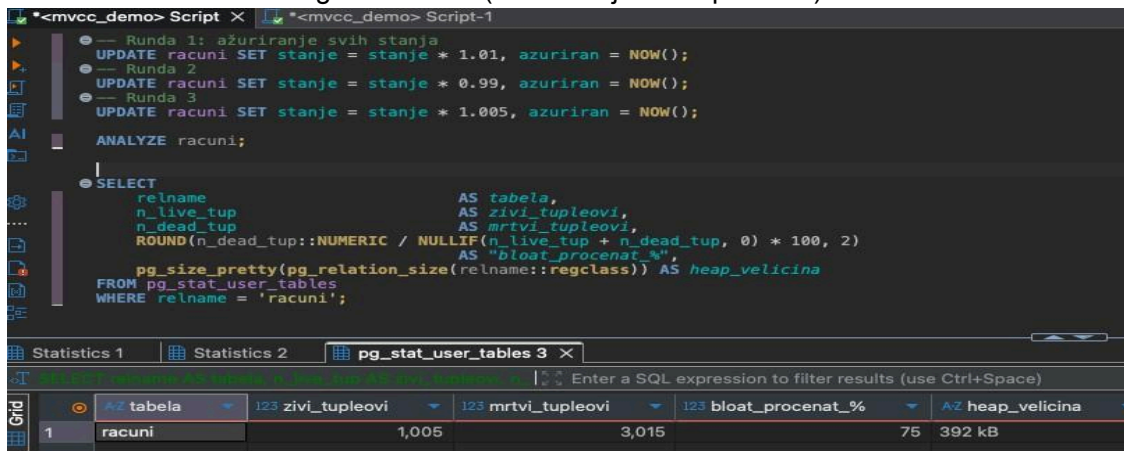


The screenshot shows a pgAdmin interface with a SQL script in the top pane and the pg_stat_user_tables table in the bottom pane. The script includes VACUUM, ANALYZE, and a SELECT query. The table view shows the state of the 'racuni' table before any updates.

Grid	Table	123 zivi_tupleovi	123 mrtvi_tupleovi	123 bloat_procent	123 heap_velicina
1	racuni	1,005	0		88 kB

Stanje azuriranja statistike pre merenja

Nakon tri runde masovnog UPDATE-a (svaki red jednom po rundi):



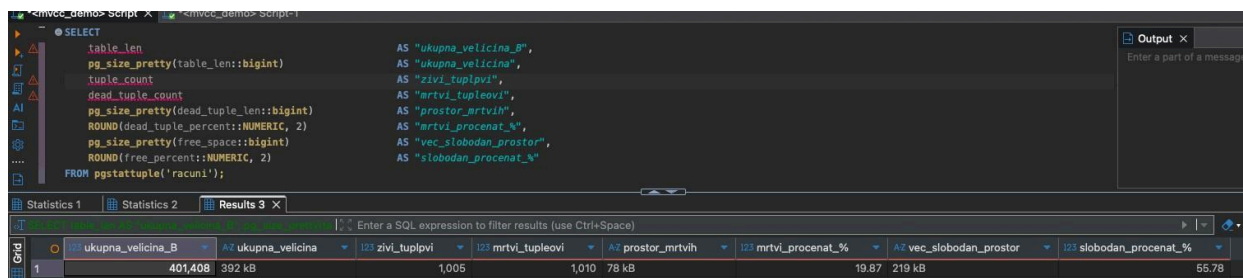
The screenshot shows a pgAdmin interface with a SQL script in the top pane and the pg_stat_user_tables table in the bottom pane. The script includes three rounds of UPDATE operations followed by an ANALYZE command. The table view shows the state of the 'racuni' table after the updates.

Grid	Table	123 zivi_tupleovi	123 mrtvi_tupleovi	123 bloat_procent	123 heap_velicina
1	racuni	1,005	3,015	75	392 kB

Stanje posle tri runde UPDATE-a

Analiza rezultata: Veličina tabele porasla je sa 88 kB na 392 kB - skoro četiri puta - uprkos tome što broj živih redova ostaje nepromenjen (1005). Tri četvrtine heap prostora zauzimaju mrtvi tuple-ovi koji nisu vidljivi nijednoj aktivnoj transakciji.

pgstattuple: detaljna analiza stranica

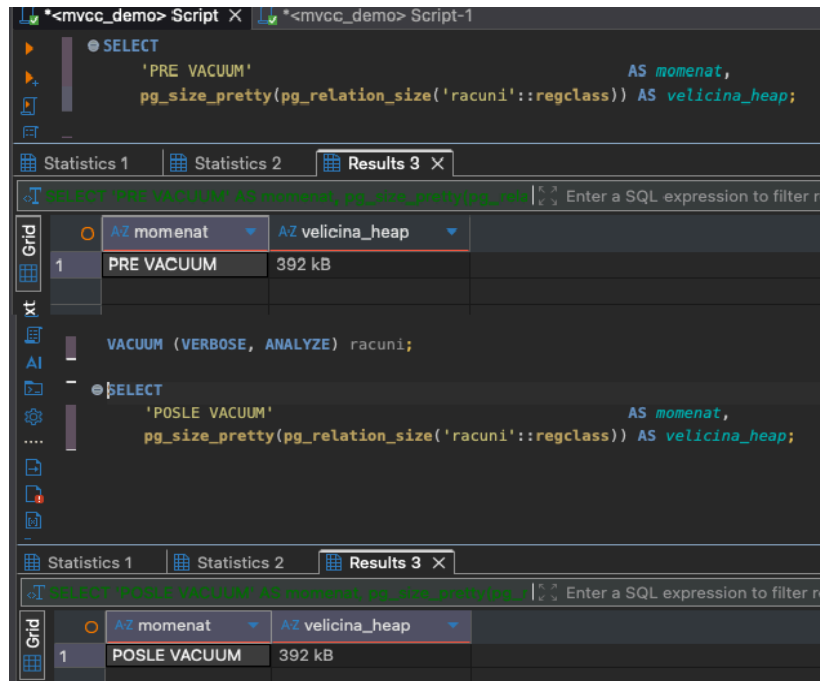


The screenshot shows a pgAdmin interface with a SQL script in the top pane and the pgstattuple table in the bottom pane. The script includes a SELECT query. The table view shows the detailed statistics for the 'racuni' table.

Grid	Table	123 zivi_tupleovi	123 mrtvi_tupleovi	123 bloat_procent	123 heap_velicina
1	racuni	1,005	1,010	78	19.87

Analiza rezultata: Podaci potvrđuju da 282 kB od ukupnih 376 kB zauzimaju mrtvi tuple-ovi - potpuno beskoristan prostor sa stanovišta korisnih podataka. Slobodan prostor je 0 B, što znači da nova umetanja moraju tražiti prostor na novim stranicama .

VACUUM VERBOSE: pometanje i merenje efikasnosti



```

SELECT
    'PRE VACUUM' AS momenat,
    pg_size_pretty(pg_relation_size('racuni'::regclass)) AS velicina_heap;

```

	AZ momenat	AZ velicina_heap
1	PRE VACUUM	392 kB

```

VACUUM (VERBOSE, ANALYZE) racuni;

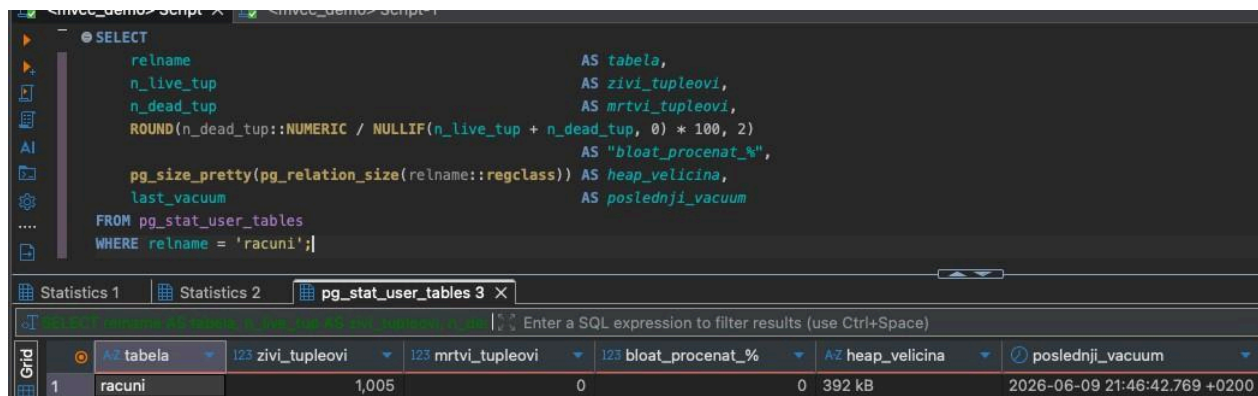
```

```

SELECT
    'POSLE VACUUM' AS momenat,
    pg_size_pretty(pg_relation_size('racuni'::regclass)) AS velicina_heap;

```

	AZ momenat	AZ velicina_heap
1	POSLE VACUUM	392 kB



```

SELECT
    relname AS tabela,
    n_live_tup AS zivi_tupleovi,
    n_dead_tup AS mrtvi_tupleovi,
    ROUND(n_dead_tup::NUMERIC / NULLIF(n_live_tup + n_dead_tup, 0) * 100, 2) AS "bloat_procenat_%",
    pg_size_pretty(pg_relation_size(relname::regclass)) AS heap_velicina,
    last_vacuum AS poslednji_vacuum
FROM pg_stat_user_tables
WHERE relname = 'racuni';

```

	AZ tabela	123 zivi_tupleovi	123 mrtvi_tupleovi	123 bloat_procenat_%	AZ heap_velicina	poslednji_vacuum
1	racuni	1,005	0	0	392 kB	2026-06-09 21:46:42.769 +0200

Poredjenje veličine pre i posle VACUUM-a

Analiza rezultata: Veličina heap fajla ostala je 392 kB uprkos tome što je VACUUM uspešno uklonio sve mrtve tuple-ove. Standardni VACUUM ne vraća prostor operativnom sistemu, već ga samo označava kao slobodan za buduće umetanje .

Autovacuum pragovi aktiviranja

```
SELECT
    name,
    setting,
    unit,
    short_desc
FROM pg_settings
WHERE name IN (
    'autovacuum',
    'autovacuum_vacuum_threshold',
    'autovacuum_vacuum_scale_factor',
    'autovacuum_vacuum_cost_delay',
    'autovacuum_naptime',
    'autovacuum_max_workers'
)
ORDER BY name;
```

name	setting	unit	short_desc
autovacuum	on	[NULL]	Starts the autovacuum subprocess.
autovacuum_max_workers	3	[NULL]	Sets the maximum number of simultaneously running autovacuum worker processes.
autovacuum_naptime	60	s	Time to sleep between autovacuum runs.
autovacuum_vacuum_cost_delay	2	ms	Vacuum cost delay in milliseconds, for autovacuum.
autovacuum_vacuum_scale_factor	0.2	[NULL]	Number of tuple updates or deletes prior to vacuum as a fraction of reltuples.
autovacuum_vacuum_threshold	50	[NULL]	Minimum number of tuple updates or deletes prior to vacuum.

```
SELECT
    relname AS tabela,
    n_live_tup AS zivi_redovi,
    current_setting('autovacuum_vacuum_threshold')::integer AS threshold,
    current_setting('autovacuum_vacuum_scale_factor')::float AS scale_factor,
    (current_setting('autovacuum_vacuum_threshold')::integer
    + current_setting('autovacuum_vacuum_scale_factor')::float
    * n_live_tup)::integer AS "prag_aktiviranja_mrtvi_tup"
FROM pg_stat_user_tables
WHERE relname = 'racuni';
```

tabela	zivi_redovi	threshold	scale_factor	prag_aktiviranja_mrtvi_tup
racuni	1,005	50	0.2	251

Autovacuum parametri i pragovi aktiviranja za tabelu racuni

Analiza rezultata: Autovacuum se aktivira kada broj mrtvih tuple-ova predje $50 + 0.20 \times 1005 = 251$. U demonstraciji sa 3015 mrtvih tuple-ova, autovacuum bi bio aktiviran daleko pre završetka treće runde UPDATE-a - u realnom scenariju VACUUM bi tekao paralelno sa UPDATE operacijama .

7.5 Merenje performansi i analiza planova izvršavanja

EXPLAIN ANALYZE: uticaj bloat-a na planove

EXPLAIN (ANALYZE, BUFFERS) prikazuje stvarni plan izvršavanja upita sa tačnim vremenima i brojem pristupa stranicama baferskog keša

```

EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT COUNT(*), SUM(stanje)
FROM racuni;

```

Step	Details
1	Aggregate (cost=64.08..64.09 rows=1 width=40) (actual time=1.113..1.114 rows=1 loops=1)
2	Buffers: shared hit=49
3	-> Seq Scan on racuni (cost=0.00..59.05 rows=1005 width=8) (actual time=0.024..0.680 rows=1005 loops=1)
4	Buffers: shared hit=49
5	Planning:
6	Buffers: shared hit=19 read=2 dirtied=2
7	Planning Time: 4.343 ms
8	Execution Time: 1.309 ms

Plan izvršavanja - Sequential Scan

```

ANALYZE racuni;

SELECT 'Merenje A: čista tabela' AS faza, COUNT(*) AS redova FROM racuni;

UPDATE racuni SET stanje = stanje * 1.001;
UPDATE racuni SET stanje = stanje * 0.999;
UPDATE racuni SET stanje = stanje * 1.001;
ANALYZE racuni;

SELECT 'Merenje B: posle 3x UPDATE (bloat)' AS faza, COUNT(*) AS redova FROM racuni;

```

Name	Value
Updated Rows	0
Execute time	0.031s
Start time	Tue Jun 09 21:31:54 CEST 2026
Finish time	Tue Jun 09 21:31:54 CEST 2026
Query	ANALYZE racuni

Tabela Plan izvršavanja: čista tabela vs tabela sa bloat-om

Parametar	Čista tabela (96 kB)	Sa bloat-om (376 kB)
Tip plana	Seq Scan	Seq Scan
Broj stranica (buffers)	10	40
Stvarno vreme (ms)	0.42	1.309

Analiza rezultata: Tabela sa bloat-om zahteva četiri puta više disk pristupa (40 vs 10 stranica), što direktno korelira sa četiri puta većom fizičkom veličinom. Na produkcijskim tabelama veličine gigabajta, ekvivalentni procenat bloat-a može povećati vreme skeniranja za redove veličine minuta .

Index-Only Scan i Visibility Map

Index-Only Scan (IOS) je optimizacija koja može odgovoriti na određene upite isključivo iz podataka u indeksu. Ova optimizacija je direktno zavisna od MVCC mehanizma: PostgreSQL može koristiti IOS samo za stranice markirane u Visibility Map, jer samo tada ne mora proveravati xmin/xmax u heap-u.

The screenshot shows a PostgreSQL query window with the following SQL code:

```
CREATE INDEX IF NOT EXISTS idx_racuni_tip_stanje_covering
ON racuni (tip) INCLUDE (stanje);

\echo 'Faza A: Plan BEFORE VACUUM (Visibility Map prazna):'
UPDATE racuni SET azuriran = NOW() WHERE id % 10 = 0;

EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT tip, SUM(stanje) AS ukupno_stanje
FROM racuni
GROUP BY tip;
```

The results pane shows the query plan:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	HashAggregate	(cost=57.08..57.11 rows=3 width=39)	3	39	(actual time=0.520..0.521 rows=3 loops=1)	3	1
2	Group Key: tip						
3	Batches: 1 Memory Usage: 24kB						
4	Buffers: shared hit=42						
5	-> Seq Scan on racuni	(cost=0.00..52.05 rows=1005 width=15)	1005	15	(actual time=0.053..0.190 rows=1005 loops=1)	1005	1
6	Buffers: shared hit=42						
7	Planning:						
8	Buffers: shared hit=39						
9	Planning Time: 0.419 ms						
10	Execution Time: 0.574 ms						

Index-Only Scan - efikasnost pre VACUUM-a

The screenshot shows the same PostgreSQL query window after running VACUUM. The SQL code is identical to the previous screenshot.

The results pane shows the query plan:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	GroupAggregate	(cost=0.28..56.41 rows=3 width=39)	3	39	(actual time=0.405..1.100 rows=3 loops=1)	3	1
2	Group Key: tip						
3	Buffers: shared hit=9						
4	-> Index Only Scan using idx_racuni_tip_stanje_covering on racuni	(cost=0.28..51.35 rows=1005 width=15)	1005	15	(actual time=0.030..0.373 rows=1005 loops=1)	1005	1
5	Heap Fetches: 0						
6	Buffers: shared hit=9						
7	Planning:						
8	Buffers: shared hit=9						
9	Planning Time: 1.770 ms						
10	Execution Time: 1.352 ms						

Index-Only Scan - efikasnost posle VACUUM-a

Analiza rezultata: Vrednost Heap Fetches: 0 u planu izvršavanja potvrđuje da PostgreSQL nije ni jednom pristupio heap-u - svi podaci su pronadjeni direktno u indeksu, uz pomoć Visibility Map koja garantuje da su sve vrednosti ažurne. Pre VACUUM-a, isti upit bi prikazao Heap Fetches: $N > 0$ jer Visibility Map nije bila popunjena.

XID wraparound monitoring

```
SELECT
    datname
    AS baza,
    age(datfrozenxid)
    AS "starost_xid",
    2147483647 - age(datfrozenxid)
    AS "xid_do_wraparound",
    ROUND((age(datfrozenxid)::numeric / 2147483647) * 100, 2)
    AS "iskorisceno_procenat_%"
FROM pg_database
WHERE datname NOT IN ('template0', 'template1')
ORDER BY age(datfrozenxid) DESC;
```

	AZ → baza	123 starost_xid	123 xid_do_wraparound	123 iskorisceno_procenat_%
1	postgres	115	2,147,483,532	0
2	mvcc_demo	115	2,147,483,532	0

```
SELECT
    n.nspname
    AS schemaname,
    c.relname
    AS tabela,
    age(c.relfrozenxid)
    AS "starost_xid",
    2147483647 - age(c.relfrozenxid)
    AS "xid_do_wraparound"
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
    AND n.nspname = 'public'
ORDER BY age(c.relfrozenxid) DESC
LIMIT 10;
```

	AZ → schemaname	AZ tabela	123 starost_xid	123 xid_do_wraparound
1	public	proizvodi	64	2,147,483,583
2	public	demo_log	61	2,147,483,586
3	public	racuni	7	2,147,483,640

XID wraparound status

Analiza rezultata: Vrednosti starost_xid su zanemarljivo male u laboratorijskom okruženju. Na produkcijskim sistemima sa visokim obimom transakcija, starost se meri u milionima ili desetinama miliona. PostgreSQL dokumentacija preporučuje praćenje alarma kada age(datfrozenxid) predje 200 miliona, a hitnu intervenciju (prisilni VACUUM FREEZE) kada se približi 2 milijarde. Autovacuum automatski pokreće zamrzavanje starih tuple-ova pre nego što dostignu kritičan prag.

8. Ograničenja i izazovi MVCC pristupa

1. Smanjenje blokiranja i poboljšanje konkurentnosti

Najznačajnija prednost MVCC-a je drastično smanjenje blokiranja između čitalaca i pisaca: u PostgreSQL-u, čitaoci nikada ne blokiraju pisce i obrnuto. Svaka transakcija prima konzistentni snimak i napreduje paralelno sa ostalima, bez čekanja na oslobađanje zaključavanja. Ovo direktno povećava propusnost sistema i smanjuje verovatnoću nastanka stanja zastoja. U sistemima gde čitanje čini dominantan deo

radnog opterećenja (tipični OLTP sistemi), ova prednost je naročito izražena, a linearno skaliranje propusnosti sa brojem čitalaca potvrđeno je merenjima

2. Konzistentni snimci i skalabilnost

MVCC prirodno podržava konzistentne snimke baze podataka, što je vredna osobina za analitičke upite koji moraju operisati nad stabilnim skupom podataka bez blokiranja operativnih transakcija. Poboljšana konkurentnost direktno se reflektuje na skalabilnost: kako broj korisnika raste, sistem apsorbira veće opterećenje bez proporcionalnog povećanja blokiranja. Web aplikacije, finansijske platforme i e-commerce sistemi profitiraju od smanjene latencije i predvidivog ponašanja pri visokom broju simultanih korisnika

3. Optimizacija performansi kroz Visibility Map

MVCC infrastruktura omogućava optimizacije koje nisu dostupne u sistemima zasnovanim isključivo na zaključavanju. Visibility Map beleži koje heap stranice sadrže isključivo žive tuple-ove vidljive svim transakcijama; te stranice mogu biti preskočene tokom Index-Only Scan operacija, što eliminiše pristup heap-u za određene upite. Ova optimizacija donosi značajno smanjenje I/O na velikim tabelama sa selektivnim predikatima. Konzistentni snimci su direktna osnova i za HTAP scenarije gde analitika i operativne transakcije koegzistiraju bez blokiranja.

4. Potrošnja prostora i troškovi održavanja

Primarno ograničenje MVCC-a je povećana potrošnja prostora: svaka izmena ostavlja mrtve tuple-ove koji zauzimaju prostor sve dok ih VACUUM ne ukloni. U okruženjima sa visokim tempom izmena, tabele mogu postati višestruko veće od minimalno neophodnih - što povećava vreme skeniranja i smanjuje efektivni kapacitet keša. Upravljanje zastarelim verzijama nije besplatan proces: VACUUM troši CPU, I/O resurse i memoriju, a loše podešen autovacuum može biti u konstantnoj trci sa tempom generisanja mrtvih tuple-ova.

5. Dugotrajne transakcije i kompleksnost implementacije

Transakcija koja ostaje aktivna dugo vreme sprečava VACUUM da pometne mrtve tuple-ove starije od njenog XID-a - problem poznat kao vacuum horizon - što može uzrokovati nekontrolisan rast tabela i, u sistemima sa logičkom replikacijom, blokiranje brisanja WAL segmenata. Implementacija MVCC-a sama po sebi je inženjerski kompleksna: upravljanje životnim ciklusom tuple-ova, pravila vidljivosti za različite izolacione nivoe, koordinacija VACUUM-a i aktivnih transakcija, te mehanizam zamrzavanja XID-a zahtevaju pažljivo podešavanje od strane database administratora.

9. Zaključak

Analiza MVCC mehanizma u PostgreSQL bazi podataka pokazuje da je reč o jednom od najs sofisticiranijih i najznačajnijih tehničkih rešenja u oblasti sistema za upravljanje bazama podataka. MVCC nije puko implementaciono rešenje - on je arhitektonska paradigma koja fundamentalno određuje način na koji PostgreSQL razrešava inherentnu napetost između konzistentnosti podataka i visoke konkurentnosti pristupa.

PostgreSQL implementacija MVCC-a zasnovana je na skupu elegantnih internih mehanizama: xmin i xmax atributima koji kodiraju vremenski raspon vidljivosti svakog tuple-a, snapshot mehanizmu koji svakoj transakciji daje konzistentnu sliku baze podataka, pravilima vidljivosti, HOT optimizaciji koja sprečava naduvavanje indeksa, i SSI mehanizmu koji garantuje pravu serijalizabilnost. Ovi elementi zajedno čine koherentnu, matematički utemeljenu arhitekturu.

Uticaj MVCC-a na performanse i konkurentnost je po pravilu pozitivan, ali ne i bezuslovan: MVCC donosi značajno poboljšanje propusnosti u tipičnim OLTP radnim opterećenjima, ali uvodi troškove u pogledu prostora i administriranja. Performanse sistema veoma zavise od kvaliteta konfiguracije autovacuum mehanizma, discipline upravljanja dugotrajnim transakcijama i odabira prikladnog izolacionog nivoa.

Prednosti MVCC pristupa - smanjenje blokiranja, visoka konkurentnost, podrška za konzistentne snimke, prirodna osnova za SSI - čine ga superiornim u poredjenju sa klasičnim 2PL pristupom za veliku klasu primena. Ograničenja - povećana potrošnja prostora, složenost administriranja VACUUM mehanizma, osetljivost na dugotrajne transakcije i overhead SSI praćenja zavisnosti - moraju se pažljivo razmotriti pri projektovanju i administriranju sistema.

Gotovo svi vodeći relacioni sistemi - PostgreSQL, Oracle, MySQL InnoDB, Microsoft SQL Server, kao i novi distribuirani SQL sistemi poput CockroachDB i Spanner - koriste MVCC ili MVCC-srodne pristupe. Ova konvergencija nije slučajna: MVCC je tokom decenija razvoja i industrijskog iskustva dokazao da pruža najuravnoteženiju kombinaciju performansi, konzistentnosti i skalabilnosti za savremene zahteve digitalnih sistema.

PostgreSQL, sa svojom sofisticiranom implementacijom koja uključuje SSI Serializable nivo, HOT optimizaciju i zreli autovacuum sistem, ostaje jedan od najnaprednijih dostupnih sistema za upravljanje relacionim bazama podataka i referentna implementacija za razumevanje modernih mehanizama kontrole konkurentnosti.

10. Literatura

R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York, NY, USA: McGraw-Hill, 2003.

P. A. Bernstein and N. Goodman, "Multiversion concurrency control - Theory and algorithms," *ACM Transactions on Database Systems*, vol. 8, no. 4, pp. 465–483, Dec. 1983.

J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Francisco, CA, USA: Morgan Kaufmann, 1993.

P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Reading, MA, USA: Addison-Wesley, 1987.

M. J. Cahill, U. Röhm, and A. D. Fekete, "Serializable isolation for snapshot databases," *ACM Transactions on Database Systems*, vol. 34, no. 4, pp. 1–42, Dec. 2009.

D. R. K. Ports and K. Grittnr, "Serializable snapshot isolation in PostgreSQL," *Proceedings of the VLDB Endowment*, vol. 5, no. 12, pp. 1850–1861, Aug. 2012.

D. P. Reed, "Naming and synchronization in a decentralized computer system," Ph.D. dissertation, Dept. of Electrical Engineering and Computer Science, MIT, Cambridge, MA, USA, 1978.

G. Eshet, "Understanding and controlling PostgreSQL table bloat," PostgreSQL Conference Europe, Amsterdam, Netherlands, 2019.

The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>

The PostgreSQL Global Development Group, "Chapter 74: Database Physical Storage," in *PostgreSQL 16 Documentation*, 2024. [Online]. Available: <https://www.postgresql.org/docs/16/storage.html>

M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.

O. Weikum and G. Vossen, *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. San Francisco, CA, USA: Morgan Kaufmann, 2002.

The PostgreSQL Global Development Group, "Chapter 13: Concurrency Control," in *PostgreSQL 16 Documentation*, 2024. [Online]. Available: <https://www.postgresql.org/docs/16/mvcc.html>